
Chapter 2 Input, Processing, and Output

Topics

2.1 Designing a Program 

2.2 Input, Processing, and Output 

2.3 Displaying Output with the `print` Function 

2.4 Comments 

2.5 Variables 

2.6 Reading Input from the Keyboard 

2.7 Performing Calculations 

2.8 More About Data Output 

2.9 Named Constants 

2.10 Introduction to Turtle Graphics 

2.1 Designing a Program

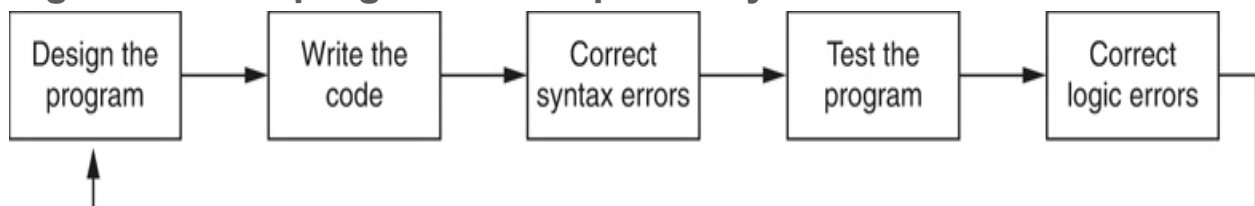
Concept:

Programs must be carefully designed before they are written. During the design process, programmers use tools such as pseudocode and flowcharts to create models of programs.

The Program Development Cycle

In [Chapter 1](#), you learned that programmers typically use high-level languages such as Python to create programs. There is much more to creating a program than writing code, however. The process of creating a program that works correctly typically requires the five phases shown in [Figure 2-1](#). The entire process is known as the *program development cycle*.

Figure 2-1 The program development cycle



Let's take a closer look at each stage in the cycle.

1. **Design the Program.** All professional programmers will tell you that a program should be carefully designed before the code is actually written. When programmers begin a new project, they should never jump right in and start writing code as the first step. They start by creating a design of the program. There are several ways to design a program, and later in this section, we will discuss some techniques that you can use to design your Python programs.
2. **Write the Code.** After designing the program, the programmer begins writing code in a high-level language such as Python. Recall from [Chapter 1](#)  that each language has its own rules, known as syntax, that must be followed when writing a program. A language's syntax rules dictate things such as how key words, operators, and punctuation characters can be used. A syntax error occurs if the programmer violates any of these rules.
3. **Correct Syntax Errors.** If the program contains a syntax error, or even a simple mistake such as a misspelled key word, the compiler or interpreter will display an error message indicating what the error is. Virtually all code contains syntax errors when it is first written, so the programmer will typically spend some time correcting these. Once all of the syntax errors and simple typing mistakes have been corrected, the program can be compiled and translated into a machine language program (or executed by an interpreter, depending on the language being used).

4. **Test the Program.** Once the code is in an executable form, it is then tested to determine whether any logic errors exist. A *logic error* is a mistake that does not prevent the program from running, but causes it to produce incorrect results.
(Mathematical mistakes are common causes of logic errors.)
5. **Correct Logic Errors.** If the program produces incorrect results, the programmer *debugs* the code. This means that the programmer finds and corrects logic errors in the program. Sometimes during this process, the programmer discovers that the program's original design must be changed. In this event, the program development cycle starts over and continues until no errors can be found.

More About the Design Process

The process of designing a program is arguably the most important part of the cycle. You can think of a program's design as its foundation. If you build a house on a poorly constructed foundation, eventually you will find yourself doing a lot of work to fix the house! A program's design should be viewed no differently. If your program is designed poorly, eventually you will find yourself doing a lot of work to fix the program.

The process of designing a program can be summarized in the following two steps:

1. Understand the task that the program is to perform.

2. Determine the steps that must be taken to perform the task.

Let's take a closer look at each of these steps.

Understand the Task That the Program Is to Perform

It is essential that you understand what a program is supposed to do before you can determine the steps that the program will perform. Typically, a professional programmer gains this understanding by working directly with the customer. We use the term *customer* to describe the person, group, or organization that is asking you to write a program. This could be a customer in the traditional sense of the word, meaning someone who is paying you to write a program. It could also be your boss, or the manager of a department within your company. Regardless of whom it is, the customer will be relying on your program to perform an important task.

To get a sense of what a program is supposed to do, the programmer usually interviews the customer. During the interview, the customer will describe the task that the program should perform, and the programmer will ask questions to uncover as many details as possible about the task. A follow-up interview is usually needed because customers rarely mention everything they want during the initial meeting, and programmers often think of additional questions.

The programmer studies the information that was gathered from the customer during the interviews and creates a list of different software requirements. A *software requirement* is simply a single task that the program must perform in order to satisfy the customer. Once the customer agrees that the list of requirements is complete, the programmer can move to the next phase.



If you choose to become a professional software developer, your customer will be anyone who asks you to write programs as part of your job. As long as you are a student, however, your customer is your instructor! In every programming class that you will take, it's practically guaranteed that your instructor will assign programming problems for you to complete. For your academic success, make sure that you understand your instructor's requirements for those assignments and write your programs accordingly.

Determine the Steps That Must Be Taken to Perform the Task

Once you understand the task that the program will perform, you begin by breaking down the task into a series of steps. This is similar to the way you would break down a task into a series of steps that another person can follow. For example, suppose someone asks you how to boil water. You might break down that task into a series of steps as follows:

1. Pour the desired amount of water into a pot.
2. Put the pot on a stove burner.
3. Turn the burner to high.
4. Watch the water until you see large bubbles rapidly rising.
When this happens, the water is boiling.

This is an example of an *algorithm*, which is a set of well-defined logical steps that must be taken to perform a task. Notice the steps in this algorithm are sequentially ordered. Step 1 should be performed before step 2, and so on. If a person follows these steps exactly as they appear, and in the correct order, he or she should be able to boil water successfully.

A programmer breaks down the task that a program must perform in a similar way. An algorithm is created, which lists all of the logical steps that must be taken. For example, suppose you have been asked to write a program to calculate and display the gross pay for an hourly paid employee. Here are the steps that you would take:

1. Get the number of hours worked.
2. Get the hourly pay rate.

3. Multiply the number of hours worked by the hourly pay rate.
4. Display the result of the calculation that was performed in step 3.

Of course, this algorithm isn't ready to be executed on the computer. The steps in this list have to be translated into code. Programmers commonly use two tools to help them accomplish this: pseudocode and flowcharts. Let's look at each of these in more detail.

Pseudocode

Because small mistakes like misspelled words and forgotten punctuation characters can cause syntax errors, programmers have to be mindful of such small details when writing code. For this reason, programmers find it helpful to write a program in pseudocode (pronounced "sue doe code") before they write it in the actual code of a programming language such as Python.

The word "pseudo" means fake, so *pseudocode* is fake code. It is an informal language that has no syntax rules and is not meant to be compiled or executed. Instead, programmers use pseudocode to create models, or "mock-ups," of programs. Because programmers don't have to worry about syntax errors while writing pseudocode, they can focus all of their attention on the program's design. Once a satisfactory design has been created with pseudocode, the pseudocode can be translated directly to actual code. Here is an

example of how you might write pseudocode for the pay calculating program that we discussed earlier:

Input the hours worked

Input the hourly pay rate

Calculate gross pay as hours worked multiplied by pay rate

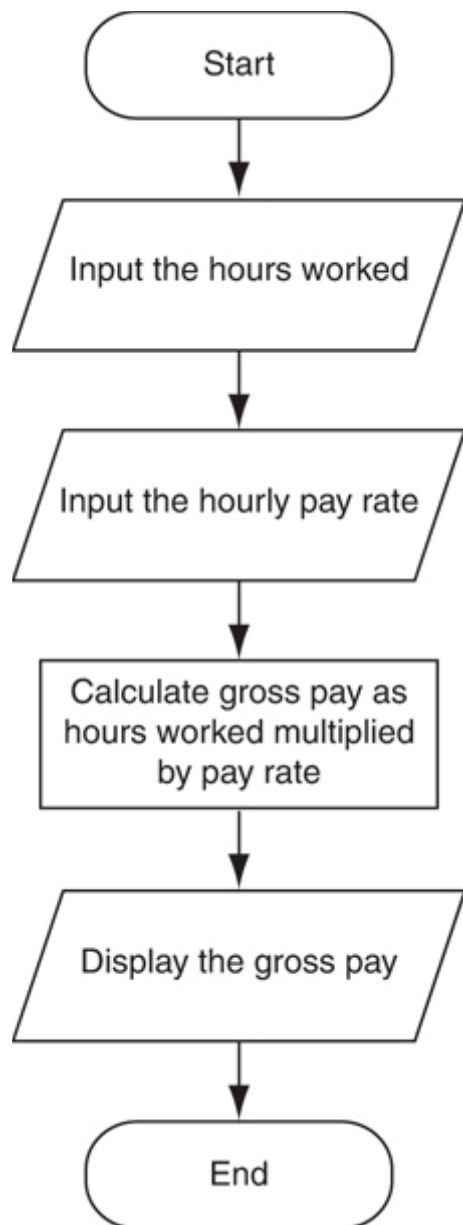
Display the gross pay

Each statement in the pseudocode represents an operation that can be performed in Python. For example, Python can read input that is typed on the keyboard, perform mathematical calculations, and display messages on the screen.

Flowcharts

Flowcharting is another tool that programmers use to design programs. A *flowchart* is a diagram that graphically depicts the steps that take place in a program. **Figure 2-2**  shows how you might create a flowchart for the pay calculating program.

Figure 2-2 Flowchart for the pay calculating program



Notice there are three types of symbols in the flowchart: ovals, parallelograms, and a rectangle. Each of these symbols represents a step in the program, as described here:

- The ovals, which appear at the top and bottom of the flowchart, are called *terminal symbols*. The *Start* terminal symbol marks the

program's starting point, and the *End* terminal symbol marks the program's ending point.

- Parallelograms are used as *input symbols* and *output symbols*. They represent steps in which the program reads input or displays output.
- Rectangles are used as *processing symbols*. They represent steps in which the program performs some process on data, such as a mathematical calculation.

The symbols are connected by arrows that represent the “flow” of the program. To step through the symbols in the proper order, you begin at the *Start* terminal and follow the arrows until you reach the *End* terminal.



Checkpoint

2.1 Who is a programmer's customer?

2.2 What is a software requirement?

2.3 What is an algorithm?

2.4 What is pseudocode?

2.5 What is a flowchart?

2.6 What do each of the following symbols mean in a flowchart?

- Oval
- Parallelogram
- Rectangle

2.2 Input, Processing, and Output

Concept:

Input is data that the program receives. When a program receives data, it usually processes it by performing some operation with it. The result of the operation is sent out of the program as output.

Computer programs typically perform the following three-step process:

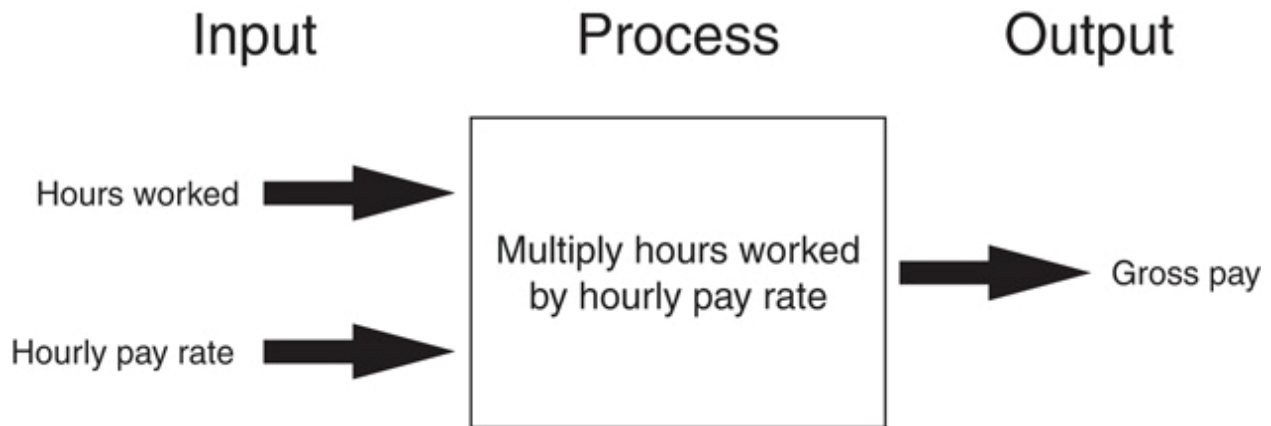
1. Input is received.
2. Some process is performed on the input.
3. Output is produced.

Input is any data that the program receives while it is running. One common form of input is data that is typed on the keyboard. Once input is received, some process, such as a mathematical calculation, is usually performed on it. The results of the process are then sent out of the program as output.

Figure 2-3  illustrates these three steps in the pay calculating program that we discussed earlier. The number of hours worked and

the hourly pay rate are provided as input. The program processes this data by multiplying the hours worked by the hourly pay rate. The results of the calculation are then displayed on the screen as output.

Figure 2-3 The input, processing, and output of the pay calculating program



In this chapter, we will discuss basic ways that you can perform input, processing, and output using Python.

2.3 Displaying Output with the `print` Function

Concept:

You use the `print` function to display output in a Python program.

A *function* is a piece of prewritten code that performs an operation. Python has numerous built-in functions that perform various operations. Perhaps the most fundamental built-in function is the `print` function, which displays output on the screen. Here is an example of a statement that executes the `print` function:



The `print` Function

```
print('Hello world')
```

In interactive mode, if you type this statement and press the Enter key, the message *Hello world* is displayed. Here is an example:

```
>>> print('Hello world') Enter  
Hello world  
>>>
```

When programmers execute a function, they say that they are *calling* the function. When you call the `print` function, you type the word `print`, followed by a set of parentheses. Inside the parentheses, you type an *argument*, which is the data that you want displayed on the screen. In the previous example, the argument is `'Hello world'`.

Notice the quote marks are not displayed when the statement executes. The quote marks simply specify the beginning and the end of the text that you wish to display.

Suppose your instructor tells you to write a program that displays your name and address on the computer screen. **Program 2-1**  shows an example of such a program, with the output that it will produce when it runs. (The line numbers that appear in a program listing in this book are *not* part of the program. We use the line numbers in our discussion to refer to parts of the program.)

Program 2-1 `(output.py)`

```
1 print('Kate Austen')
2 print('123 Full Circle Drive')
3 print('Asheville, NC 28899')
```

Program Output

```
Kate Austen
123 Full Circle Drive
Asheville, NC 28899
```

It is important to understand that the statements in this program execute in the order that they appear, from the top of the program to the bottom. When you run this program, the first statement will execute, followed by the second statement, and followed by the third statement.

Strings and String Literals

Programs almost always work with data of some type. For example, **Program 2-1**  uses the following three pieces of data:

```
'Kate Austen'
```



```
'123 Full Circle Drive'
```

```
'Asheville, NC 28899'
```

These pieces of data are sequences of characters. In programming terms, a sequence of characters that is used as data is called a *string*. When a string appears in the actual code of a program, it is called a *string literal*. In Python code, string literals must be enclosed in quote marks. As mentioned earlier, the quote marks simply mark where the string data begins and ends.

In Python, you can enclose string literals in a set of single-quote marks (') or a set of double-quote marks ("). The string literals in **Program 2-1** are enclosed in single-quote marks, but the program could also be written as shown in **Program 2-2**.

Program 2-2 (*double_quotes.py*)

```
1 print("Kate Austen")
2 print("123 Full Circle Drive")
3 print("Asheville, NC 28899")
```

Program Output

```
Kate Austen
123 Full Circle Drive
Asheville, NC 28899
```

If you want a string literal to contain either a single-quote or an apostrophe as part of the string, you can enclose the string literal in double-quote marks. For example, [Program 2-3](#)  prints two strings that contain apostrophes.

Program 2-3 (*apostrophe.py*)

```
1 print("Don't fear!")  
2 print("I'm here!")
```

Program Output

```
Don't fear!  
I'm here!
```

Likewise, you can use single-quote marks to enclose a string literal that contains double-quotes as part of the string. [Program 2-4](#)  shows an example.

Program 2-4 (*display_quote.py*)

```
1 print('Your assignment is to read "Hamlet" by tomorrow.')
```

Program Output

```
Your assignment is to read "Hamlet" by tomorrow.
```

Python also allows you to enclose string literals in triple quotes (either `"""` or `'''`). Triple-quoted strings can contain both single quotes and double quotes as part of the string. The following statement shows an example:

```
print("""I'm reading "Hamlet" tonight.""")
```

This statement will print

```
I'm reading "Hamlet" tonight.
```

Triple quotes can also be used to surround multiline strings, something for which single and double quotes cannot be used. Here is an example:

```
print("""One  
Two  
Three""")
```

This statement will print

```
One
```

```
Two
```

```
Three
```



Checkpoint

2.7 Write a statement that displays your name.

2.8 Write a statement that displays the following text:

```
Python's the best!
```

2.9 Write a statement that displays the following text:

```
The cat said "meow."
```

2.4 Comments

Concept:

Comments are notes of explanation that document lines or sections of a program. Comments are part of the program, but the Python interpreter ignores them. They are intended for people who may be reading the source code.

Comments are short notes placed in different parts of a program, explaining how those parts of the program work. Although comments are a critical part of a program, they are ignored by the Python interpreter. Comments are intended for any person reading a program's code, not the computer.

In Python, you begin a comment with the `#` character. When the Python interpreter sees a `#` character, it ignores everything from that character to the end of the line. For example, look at [Program 2-5](#). Lines 1 and 2 are comments that briefly explain the program's purpose.

Program 2-5 `(comment1.py)`

```
1 # This program displays a person's
2 # name and address.
3 print('Kate Austen')
4 print('123 Full Circle Drive')
5 print('Asheville, NC 28899')
```

Program Output

```
Kate Austen
123 Full Circle Drive
Asheville, NC 28899
```

Programmers commonly write end-line comments in their code. An *end-line comment* is a comment that appears at the end of a line of code. It usually explains the statement that appears in that line.

Program 2-6  shows an example. Each line ends with a comment that briefly explains what the line does.

Program 2-6 (comment2.py)

```
1 print('Kate Austen')           # Display the name.
2 print('123 Full Circle Drive')  # Display the address.
3 print('Asheville, NC 28899')    # Display the city,
state, and ZIP.
```

Program Output

```
Kate Austen  
123 Full Circle Drive  
Asheville, NC 28899
```

As a beginning programmer, you might be resistant to the idea of liberally writing comments in your programs. After all, it can seem more productive to write code that actually does something! It is crucial that you take the extra time to write comments, however. They will almost certainly save you and others time in the future when you have to modify or debug the program. Large and complex programs can be almost impossible to read and understand if they are not properly commented.

2.5 Variables

Concept:

A variable is a name that represents a value stored in the computer's memory.

Programs usually store data in the computer's memory and perform operations on that data. For example, consider the typical online shopping experience: you browse a website and add the items that you want to purchase to the shopping cart. As you add items to the shopping cart, data about those items is stored in memory. Then, when you click the checkout button, a program running on the website's computer calculates the cost of all the items you have in your shopping cart, applicable sales taxes, shipping costs, and the total of all these charges. When the program performs these calculations, it stores the results in the computer's memory.

Programs use variables to access and manipulate data that is stored in memory. A *variable* is a name that represents a value in the computer's memory. For example, a program that calculates the sales tax on a purchase might use the variable name `tax` to represent that value in memory. And a program that calculates the distance between

two cities might use the variable name `distance` to represent that value in memory. When a variable represents a value in the computer's memory, we say that the variable *references* the value.

Creating Variables with Assignment Statements

You use an *assignment statement* to create a variable and make it reference a piece of data. Here is an example of an assignment statement:

```
age = 25
```

After this statement executes, a variable named `age` will be created, and it will reference the value 25. This concept is shown in **Figure 2-4**. In the figure, think of the value 25 as being stored somewhere in the computer's memory. The arrow that points from `age` to the value 25 indicates that the name `age` references the value.

Figure 2-4 The `age` variable references the value 25

`age` → 25

An assignment statement is written in the following general format:

```
variable = expression
```

The equal sign (=) is known as the *assignment operator*. In the general format, `variable` is the name of a variable and `expression` is a value, or any piece of code that results in a value. After an assignment statement executes, the variable listed on the left side of the = operator will reference the value given on the right side of the = operator.

To experiment with variables, you can type assignment statements in interactive mode, as shown here:

```
>>> width = 10   
>>> length = 5   
>>>
```

The first statement creates a variable named `width` and assigns it the value 10. The second statement creates a variable named `length` and assigns it the value 5. Next, you can use the `print` function to display the values referenced by these variables, as shown here:

```
>>> print(width)   
10  
>>> print(length)   
5
```

```
>>>
```

When you pass a variable as an argument to the `print` function, you do not enclose the variable name in quote marks. To demonstrate why, look at the following interactive session:

```
>>> print('width') Enter
width
>>> print(width) Enter
10
>>>
```

In the first statement, you passed `'width'` as an argument to the `print` function, and the function printed the string `width`. In the second statement, you passed `width` (with no quote marks) as an argument to the `print` function, and the function displayed the value referenced by the `width` variable.

In an assignment statement, the variable that is receiving the assignment must appear on the left side of the `=` operator. As shown in the following interactive session, an error occurs if the item on the left side of the `=` operator is not a variable:

```
>>> 25 = age Enter
SyntaxError: can't assign to literal
```

```
>>>
```

The code in [Program 2-7](#) demonstrates a variable. Line 2 creates a variable named `room` and assigns it the value 503. The statements in lines 3 and 4 display a message. Notice line 4 displays the value that is referenced by the `room` variable.

Program 2-7 (*variable_demo.py*)

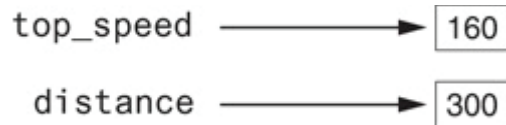
```
1  # This program demonstrates a variable.  
2  room = 503  
3  print('I am staying in room number')  
4  print(room)
```

Program Output

```
I am staying in room number  
503
```

[Program 2-8](#) shows a sample program that uses two variables. Line 2 creates a variable named `top_speed`, assigning it the value 160. Line 3 creates a variable named `distance`, assigning it the value 300. This is illustrated in [Figure 2-5](#).

Figure 2-5 Two variables



Program 2-8 *(variable_demo2.py)*

```
1 # Create two variables: top_speed and distance.
2 top_speed = 160
3 distance = 300
4
5 # Display the values referenced by the variables.
6 print('The top speed is')
7 print(top_speed)
8 print('The distance traveled is')
9 print(distance)
```

Program Output

```
The top speed is
160
The distance traveled is
300
```



Warning!

You cannot use a variable until you have assigned a value to it. An error will occur if you try to perform an operation on a variable, such as printing it, before it has been assigned a value.

Sometimes a simple typing mistake will cause this error. One example is a misspelled variable name, as shown here:

```
temperature = 74.5 # Create a variable  
print(tempereture) # Error! Misspelled variable name
```

In this code, the variable `temperature` is created by the assignment statement. The variable name is spelled differently in the `print` statement, however, which will cause an error.

Another example is the inconsistent use of uppercase and lowercase letters in a variable name. Here is an example:

```
temperature = 74.5 # Create a variable  
print(Temperature) # Error! Inconsistent use of case
```

In this code, the variable `temperature` (in all lowercase letters) is created by the assignment statement. In the `print` statement, the name `Temperature` is spelled with an uppercase T. This will cause an error because variable names are case sensitive in Python.

Variable Naming Rules

Although you are allowed to make up your own names for variables, you must follow these rules:

- You cannot use one of Python's key words as a variable name. (See [Table 1-2](#) for a list of the key words.)
- A variable name cannot contain spaces.
- The first character must be one of the letters a through z, A through Z, or an underscore character (_).
- After the first character you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
- Uppercase and lowercase characters are distinct. This means the variable name `ItemsOrdered` is not the same as `itemsordered`.

In addition to following these rules, you should always choose names for your variables that give an indication of what they are used for. For example, a variable that holds the temperature might be named `temperature`, and a variable that holds a car's speed might be named `speed`. You may be tempted to give variables names such as `x` and `b2`, but names like these give no clue as to what the variable's purpose is.

Because a variable's name should reflect the variable's purpose, programmers often find themselves creating names that are made of multiple words. For example, consider the following variable names:

```
grosspay  
payrate  
hotdogssoldtoday
```

Unfortunately, these names are not easily read by the human eye because the words aren't separated. Because we can't have spaces in variable names, we need to find another way to separate the words in a multiword variable name and make it more readable to the human eye.

One way to do this is to use the underscore character to represent a space. For example, the following variable names are easier to read than those previously shown:

```
gross_pay  
pay_rate  
hot_dogs_sold_today
```

This style of naming variables is popular among Python programmers, and is the style we will use in this book. There are other popular styles, however, such as the *camelCase* naming convention. camelCase names are written in the following manner:

- The variable name begins with lowercase letters.
- The first character of the second and subsequent words is written in uppercase.

For example, the following variable names are written in camelCase:

```
grossPay
payRate
hotDogsSoldToday
```



Note:

This style of naming is called camelCase because the uppercase characters that appear in a name may suggest a camel's humps.

Table 2-1  lists several sample variable names and indicates whether each is legal or illegal in Python.

Table 2-1 Sample variable names

Variable Name	Legal or Illegal?
<code>units_per_day</code>	Legal

<code>dayOfWeek</code>	Legal
<code>3dGraph</code>	Illegal. Variable names cannot begin with a digit.
<code>June1997</code>	Legal
<code>Mixture#3</code>	Illegal. Variable names may only use letters, digits, or underscores.

Displaying Multiple Items with the `print` Function

If you refer to [Program 2-7](#), you will see that we used the following two statements in lines 3 and 4:

```
print('I am staying in room number')
print(room)
```

We called the `print` function twice because we needed to display two pieces of data. Line 3 displays the string literal `'I am staying in room number'`, and line 4 displays the value referenced by the `room` variable.

This program can be simplified, however, because Python allows us to display multiple items with one call to the `print` function. We simply have to separate the items with commas as shown in [Program 2-9](#).

Program 2-9 `(variable_demo3.py)`

```
1 # This program demonstrates a variable.  
2 room = 503  
3 print('I am staying in room number', room)
```

Program Output

```
I am staying in room number 503
```

In line 3, we passed two arguments to the `print` function. The first argument is the string literal `'I am staying in room number'`, and the second argument is the `room` variable. When the `print` function executed, it displayed the values of the two arguments in the order that we passed them to the function. Notice the `print` function automatically printed a space separating the two items. When multiple arguments are passed to the `print` function, they are automatically separated by a space when they are displayed on the screen.

Variable Reassignment

Variables are called “variable” because they can reference different values while a program is running. When you assign a value to a variable, the variable will reference that value until you assign it a

different value. For example, look at [Program 2-10](#). The statement in line 3 creates a variable named `dollars` and assigns it the value 2.75. This is shown in the top part of [Figure 2-6](#). Then, the statement in line 8 assigns a different value, 99.95, to the `dollars` variable. The bottom part of [Figure 2-6](#) shows how this changes the `dollars` variable. The old value, 2.75, is still in the computer's memory, but it can no longer be used because it isn't referenced by a variable. When a value in memory is no longer referenced by a variable, the Python interpreter automatically removes it from memory through a process known as *garbage collection*.

Figure 2-6 Variable reassignment in [Program 2-10](#)

The dollars variable after line 3 executes.



The dollars variable after line 8 executes.



Program 2-10 (`variable_demo4.py`)

```
1 # This program demonstrates variable reassignment.
2 # Assign a value to the dollars variable.
3 dollars = 2.75
4 print('I have', dollars, 'in my account.')
5
```

```
6 # Reassign dollars so it references
7 # a different value.
8 dollars = 99.95
9 print('But now I have', dollars, 'in my account!')
```

Program Output

```
I have 2.75 in my account.
But now I have 99.95 in my account!
```

Numeric Data Types and Literals

In [Chapter 1](#), we discussed the way that computers store data in memory. (See [Section 1.3](#)) You might recall from that discussion that computers use a different technique for storing real numbers (numbers with a fractional part) than for storing integers. Not only are these types of numbers stored differently in memory, but similar operations on them are carried out in different ways.

Because different types of numbers are stored and manipulated in different ways, Python uses *data types* to categorize values in memory. When an integer is stored in memory, it is classified as an `int`, and when a real number is stored in memory, it is classified as a `float`.

Let's look at how Python determines the data type of a number. Several of the programs that you have seen so far have numeric data written into their code. For example, the following statement, which appears in [Program 2-9](#), has the number 503 written into it:

```
room = 503
```

This statement causes the value 503 to be stored in memory, and it makes the `room` variable reference it. The following statement, which appears in [Program 2-10](#), has the number 2.75 written into it:

```
dollars = 2.75
```

This statement causes the value 2.75 to be stored in memory, and it makes the `dollars` variable reference it. A number that is written into a program's code is called a *numeric literal*. When the Python interpreter reads a numeric literal in a program's code, it determines its data type according to the following rules:

- A numeric literal that is written as a whole number with no decimal point is considered an `int`. Examples are `7`, `124`, and `-9`.
- A numeric literal that is written with a decimal point is considered a `float`. Examples are `1.5`, `3.14159`, and `5.0`.

So, the following statement causes the number 503 to be stored in memory as an `int`:

```
room = 503
```

And the following statement causes the number 2.75 to be stored in memory as a `float`:

```
dollars = 2.75
```

When you store an item in memory, it is important for you to be aware of the item's data type. As you will see, some operations behave differently depending on the type of data involved, and some operations can only be performed on values of a specific data type.

As an experiment, you can use the built-in `type` function in interactive mode to determine the data type of a value. For example, look at the following session:

```
>>> type(1)   
<class 'int'>  
>>>
```

In this example, the value 1 is passed as an argument to the `type` function. The message that is displayed on the next line, `<class 'int'>`, indicates that the value is an `int`. Here is another example:

```
>>> type(1.0) Enter  
<class 'float'>  
>>>
```

In this example, the value 1.0 is passed as an argument to the `type` function. The message that is displayed on the next line, `<class 'float'>`, indicates that the value is a `float`.



Warning!

You cannot write currency symbols, spaces, or commas in numeric literals. For example, the following statement will cause an error:

```
value = $4,567.99 # Error!
```

This statement must be written as:

```
value = 4567.99 # Correct
```


Storing Strings with the `str` Data Type

In addition to the `int` and `float` data types, Python also has a data type named `str`, which is used for storing strings in memory. The code in [Program 2-11](#) shows how strings can be assigned to variables.

Program 2-11 (string_variable.py)

```
1 # Create variables to reference two strings.
2 first_name = 'Kathryn'
3 last_name = 'Marino'
4
5 # Display the values referenced by the variables.
6 print(first_name, last_name)
```

Program Output

```
Kathryn Marino
```

Reassigning a Variable to a Different Type

Keep in mind that in Python, a variable is just a name that refers to a piece of data in memory. It is a mechanism that makes it easy for you, the programmer, to store and retrieve data. Internally, the Python interpreter keeps track of the variable names that you create and the pieces of data to which those variable names refer. Any time you need to retrieve one of those pieces of data, you simply use the variable name that refers to it.

A variable in Python can refer to items of any type. After a variable has been assigned an item of one type, it can be reassigned an item of a different type. To demonstrate, look at the following interactive session. (We have added line numbers for easier reference.)

```
1  >>> x = 99   
2  >>> print(x)   
3  99  
4  >>> x = 'Take me to your leader'   
5  >>> print(x)   
6  Take me to your leader.  
7  >>>
```

The statement in line 1 creates a variable named `x` and assigns it the `int` value 99. **Figure 2-7** shows how the variable `x` references the value 99 in memory. The statement in line 2 calls the `print` function, passing `x` as an argument. The output of the `print` function is shown in line 3. Then, the statement in line 4 assigns a string to the `x` variable. After this statement executes, the `x` variable no longer refers to an `int`, but to the string `'Take me to your leader'`. This is shown in **Figure 2-8**. Line 5 calls the `print` function again, passing `x` as an argument. Line 6 shows the `print` function's output.

Figure 2-7 The variable `x` references an integer



Figure 2-8 The variable `x` references a string



Checkpoint

2.10 What is a variable?

2.11 Which of the following are illegal variable names in Python, and why?

```
x
99bottles
july2009
theSalesFigureForFiscalYear
```

```
r&d  
grade_report
```

2.12 Is the variable name `Sales` the same as `sales`? Why or why not?

2.13 Is the following assignment statement valid or invalid? If it is invalid, why?

```
72 = amount
```

2.14 What will the following code display?

```
val = 99  
print('The value is', 'val')
```

2.15 Look at the following assignment statements:

```
value1 = 99  
value2 = 45.9  
value3 = 7.0  
value4 = 7  
value5 = 'abc'
```

After these statements execute, what is the Python data type of the values referenced by each variable?

2.16 What will be displayed by the following program?

```
my_value = 99
```

```
my_value = 0
```

```
print(my_value)
```

2.6 Reading Input from the Keyboard

Concept:

Programs commonly need to read input typed by the user on the keyboard. We will use the Python functions to do this.

Most of the programs that you will write will need to read input and then perform an operation on that input. In this section, we will discuss a basic input operation: reading data that has been typed on the keyboard. When a program reads data from the keyboard, usually it stores that data in a variable so it can be used later by the program.



Reading Input from the Keyboard

In this book, we use Python's built-in `input` function to read input from the keyboard. The `input` function reads a piece of data that has been entered at the keyboard and returns that piece of data, as a string, back to the program. You normally use the `input` function in an assignment statement that follows this general format:

```
variable = input(prompt)
```

In the general format, `prompt` is a string that is displayed on the screen. The string's purpose is to instruct the user to enter a value; `variable` is the name of a `variable` that references the data that was entered on the keyboard. Here is an example of a statement that uses the `input` function to read data from the keyboard:

```
name = input('What is your name? ')
```

When this statement executes, the following things happen:

- The string `'What is your name? '` is displayed on the screen.
- The program pauses and waits for the user to type something on the keyboard and then to press the Enter key.
- When the Enter key is pressed, the data that was typed is returned as a string and assigned to the `name` variable.

To demonstrate, look at the following interactive session:

```
>>> name = input('What is your name? ')   
What is your name? Holly   
>>> print(name)   
Holly  
>>>
```

When the first statement was entered, the interpreter displayed the prompt `'What is your name? '` and waited for the user to enter some data. The user entered *Holly* and pressed the Enter key. As a result, the string `'Holly'` was assigned to the `name` variable. When the second statement was entered, the interpreter displayed the value referenced by the `name` variable.

Program 2-12  shows a complete program that uses the `input` function to read two strings as input from the keyboard.

Program 2-12 `(string_input.py)`

```
1  # Get the user's first name.  
2  first_name = input('Enter your first name: ')  
3  
4  # Get the user's last name.  
5  last_name = input('Enter your last name: ')  
6  
7  # Print a greeting to the user.  
8  print('Hello', first_name, last_name)
```


Program Output (with input shown in bold)

```
Enter your first name: Vinny   
Enter your last name: Brown   
Hello Vinny Brown
```

Take a closer look in line 2 at the string we used as a prompt:

```
'Enter your first name: '
```

Notice the last character in the string, inside the quote marks, is a space. The same is true for the following string, used as prompt in line 5:

```
'Enter your last name: '
```

We put a space character at the end of each string because the `input` function does not automatically display a space after the prompt. When the user begins typing characters, they appear on the screen immediately after the prompt. Making the last character in the prompt a space visually separates the prompt from the user's input on the screen.

Reading Numbers with the `input` Function

The `input` function always returns the user's input as a string, even if the user enters numeric data. For example, suppose you call the `input` function, type the number 72, and press the Enter key. The value that is returned from the `input` function is the string `'72'`. This can be a problem if you want to use the value in a math operation. Math operations can be performed only on numeric values, not strings.

Fortunately, Python has built-in functions that you can use to convert a string to a numeric type. [Table 2-2](#) summarizes two of these functions.

Table 2-2 Data conversion functions

Function	Description
<code>int(item)</code>	You pass an argument to the <code>int()</code> function and it returns the argument's value converted to an <code>int</code> .
<code>float(item)</code>	You pass an argument to the <code>float()</code> function and it returns the argument's value converted to a <code>float</code> .

For example, suppose you are writing a payroll program and you want to get the number of hours that the user has worked. Look at the

following code:

```
string_value = input('How many hours did you work? ')\nhours = int(string_value)
```

The first statement gets the number of hours from the user and assigns that value as a string to the `string_value` variable. The second statement calls the `int()` function, passing `string_value` as an argument. The value referenced by `string_value` is converted to an `int` and assigned to the `hours` variable.

This example illustrates how the `int()` function works, but it is inefficient because it creates two variables: one to hold the string that is returned from the `input` function, and another to hold the integer that is returned from the `int()` function. The following code shows a better approach. This one statement does all the work that the previously shown two statements do, and it creates only one variable:

```
hours = int(input('How many hours did you work? '))
```

This one statement uses *nested function* calls. The value that is returned from the `input` function is passed as an argument to the `int()` function. This is how it works:

- It calls the `input` function to get a value entered at the keyboard.
- The value that is returned from the `input` function (a string) is passed as an argument to the `int()` function.
- The `int` value that is returned from the `int()` function is assigned to the `hours` variable.

After this statement executes, the `hours` variable is assigned the value entered at the keyboard, converted to an `int`.

Let's look at another example. Suppose you want to get the user's hourly pay rate. The following statement prompts the user to enter that value at the keyboard, converts the value to a `float`, and assigns it to the `pay_rate` variable:

```
pay_rate = float(input('What is your hourly pay rate? '))
```

This is how it works:

- It calls the `input` function to get a value entered at the keyboard.
- The value that is returned from the `input` function (a string) is passed as an argument to the `float()` function.
- The `float` value that is returned from the `float()` function is assigned to the `pay_rate` variable.

After this statement executes, the `pay_rate` variable is assigned the value entered at the keyboard, converted to a `float`.

Program 2-13  shows a complete program that uses the `input` function to read a string, an `int`, and a `float`, as input from the keyboard.

Program 2-13 (`input.py`)

```
1  # Get the user's name, age, and income.
2  name = input('What is your name? ')
3  age = int(input('What is your age? '))
4  income = float(input('What is your income? '))
5
6  # Display the data.
7  print('Here is the data you entered:')
8  print('Name:', name)
9  print('Age:', age)
10 print('Income:', income)
```

Program Output (with input shown in bold)

```
What is your name? Chris 
What is your age? 25 
What is your income? 75000.0
Here is the data you entered:
```

```
Name: Chris
Age: 25
Income: 75000.0
```

Let's take a closer look at the code:

- Line 2 prompts the user to enter his or her name. The value that is entered is assigned, as a string, to the `name` variable.
- Line 3 prompts the user to enter his or her age. The value that is entered is converted to an `int` and assigned to the `age` variable.
- Line 4 prompts the user to enter his or her income. The value that is entered is converted to a `float` and assigned to the `income` variable.
- Lines 7 through 10 display the values that the user entered.

The `int()` and `float()` functions work only if the item that is being converted contains a valid numeric value. If the argument cannot be converted to the specified data type, an error known as an exception occurs. An *exception* is an unexpected error that occurs while a program is running, causing the program to halt if the error is not properly dealt with. For example, look at the following interactive mode session:

```
>>> age = int(input('What is your age? '))
What is your age? xyz
Traceback (most recent call last):
  File "<pyshell#81>", line 1, in <module>
```

```
age = int(input('What is your age? '))  
ValueError: invalid literal for int() with base 10: 'xyz'  
>>>
```



Note:

In this section, we mentioned the user. The *user* is simply any hypothetical person that is using a program and providing input for it. The user is sometimes called the *end user*.



Checkpoint

2.17 You need the user of a program to enter a customer's last name. Write a statement that prompts the user to enter this data and assigns the input to a variable.

2.18 You need the user of a program to enter the amount of sales for the week. Write a statement that prompts the user to enter this data and assigns the input to a variable.

2.7 Performing Calculations

Concept:

Python has numerous operators that can be used to perform mathematical calculations.

Most real-world algorithms require calculations to be performed. A programmer's tools for performing calculations are *math operators*. **Table 2-3**  lists the math operators that are provided by the Python language.

Table 2-3 Python math operators

Symbol	Operation	Description
+	Addition	Adds two numbers
-	Subtraction	Subtracts one number from another
*	Multiplication	Multiplies one number by another
/	Division	Divides one number by another and gives the result as a floating-point number
//	Integer	Divides one number by another and gives the result as a

	division	whole number
<code>%</code>	Remainder	Divides one number by another and gives the remainder
<code>**</code>	Exponent	Raises a number to a power

Programmers use the operators shown in [Table 2-3](#)  to create math expressions. A *math expression* performs a calculation and gives a value. The following is an example of a simple math expression:

```
12 + 2
```

The values on the right and left of the + operator are called *operands*. These are values that the + operator adds together. If you type this expression in interactive mode, you will see that it gives the value 14:

```
>>> 12 + 2 
```

```
14
```

```
>>>
```

Variables may also be used in a math expression. For example, suppose we have two variables named `hours` and `pay_rate`. The following math expression uses the `*` operator to multiply the value referenced by the `hours` variable by the value referenced by the `pay_rate` variable:

```
hours * pay_rate
```

When we use a math expression to calculate a value, normally we want to save that value in memory so we can use it again in the program. We do this with an assignment statement. [Program 2-14](#)  shows an example.

Program 2-14 `(simple_math.py)`

```
1  # Assign a value to the salary variable.
2  salary = 2500.0
3
4  # Assign a value to the bonus variable.
5  bonus = 1200.0
6
7  # Calculate the total pay by adding salary
8  # and bonus. Assign the result to pay.
9  pay = salary + bonus
10
11 # Display the pay.
12 print('Your pay is', pay)
```

Program Output

```
Your pay is 3700.0
```

Line 2 assigns 2500.0 to the `salary` variable, and line 5 assigns 1200.0 to the `bonus` variable. Line 9 assigns the result of the expression `salary + bonus` to the `pay` variable. As you can see from the program output, the `pay` variable holds the value 3700.0.



In the

Spotlight: Calculating a Percentage

If you are writing a program that works with a percentage, you have to make sure that the percentage's decimal point is in the correct location before doing any math with the percentage.

This is especially true when the user enters a percentage as input. Most users enter the number 50 to mean 50 percent, 20 to mean 20 percent, and so forth. Before you perform any calculations with such a percentage, you have to divide it by 100 to move its decimal point two places to the left.

Let's step through the process of writing a program that calculates a percentage. Suppose a retail business is planning to have a storewide sale where the prices of all items will be 20 percent off. We have been asked to write a program to calculate the sale price of an item after the discount is subtracted. Here is the algorithm:

1. Get the original price of the item.
2. Calculate 20 percent of the original price. This is the amount of the discount.

3. Subtract the discount from the original price. This is the sale price.
4. Display the sale price.

In step 1, we get the original price of the item. We will prompt the user to enter this data on the keyboard. In our program we will use the following statement to do this. Notice the value entered by the user will be stored in a variable named

`original_price`.

```
original_price = float(input("Enter the item's original  
price: "))
```

In step 2, we calculate the amount of the discount. To do this, we multiply the original price by 20 percent. The following statement performs this calculation and assigns the result to the `discount` variable:

```
discount = original_price * 0.2
```

In step 3, we subtract the discount from the original price. The following statement does this calculation and stores the result in the `sale_price` variable:

```
sale_price = original_price - discount
```

Last, in step 4, we will use the following statement to display the sale price:

```
print('The sale price is', sale_price)
```

Program 2-15  shows the entire program, with example output.

Program 2-15 (sale_price.py)

```
1  # This program gets an item's original price and
2  # calculates its sale price, with a 20% discount.
3
4  # Get the item's original price.
5  original_price = float(input("Enter the item's
original price: "))
6
7  # Calculate the amount of the discount.
8  discount = original_price * 0.2
9
10 # Calculate the sale price.
11 sale_price = original_price - discount
12
13 # Display the sale price.
14 print('The sale price is', sale_price)
```

Program Output (with input shown in bold)

```
Enter the item's original price: 100.00 Enter  
The sale price is 80.0
```

Floating-Point and Integer Division

Notice in [Table 2-3](#) that Python has two different division operators. The `/` operator performs floating-point division, and the `//` operator performs integer division. Both operators divide one number by another. The difference between them is that the `/` operator gives the result as a floating-point value, and the `//` operator gives the result as a whole number. Let's use the interactive mode interpreter to demonstrate:

```
>>> 5 / 2 Enter  
2.5  
>>>
```

In this session, we used the `/` operator to divide 5 by 2. As expected, the result is 2.5. Now let's use the `//` operator to perform integer division:

```
>>> 5 // 2 
```

```
2
```

```
>>>
```

As you can see, the result is 2. The `//` operator works like this:

- When the result is positive, it is *truncated*, which means that its fractional part is thrown away.
- When the result is negative, it is rounded *away from zero* to the nearest integer.

The following interactive session demonstrates how the `//` operator works when the result is negative:

```
>>> -5 // 2 
```

```
-3
```

```
>>>
```

Operator Precedence

You can write statements that use complex mathematical expressions involving several operators. The following statement assigns the sum of 17, the variable `x`, 21, and the variable `y` to the variable `answer`:

```
answer = 17 + x + 21 + y
```

Some expressions are not that straightforward, however. Consider the following statement:

```
outcome = 12.0 + 6.0 / 3.0
```

What value will be assigned to `outcome`? The number 6.0 might be used as an operand for either the addition or division operator. The `outcome` variable could be assigned either 6.0 or 14.0, depending on when the division takes place. Fortunately, the answer can be predicted because Python follows the same order of operations that you learned in math class.

First, operations that are enclosed in parentheses are performed first. Then, when two operators share an operand, the operator with the higher *precedence* is applied first. The precedence of the math operators, from highest to lowest, are:

1. Exponentiation: `**`
2. Multiplication, division, and remainder: `*` `/` `//` `%`
3. Addition and subtraction: `+` `-`

Notice the multiplication (`*`), floating-point division (`/`), integer division (`//`), and remainder (`%`) operators have the same precedence. The addition (`+`) and subtraction (`-`) operators also have the same

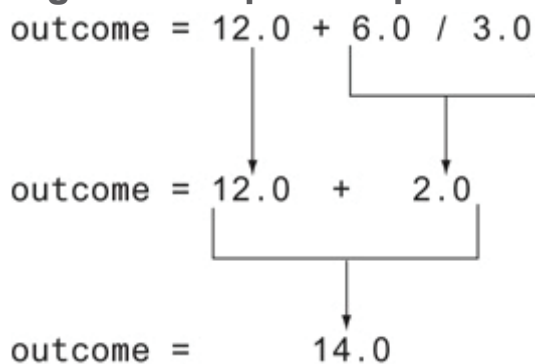
precedence. When two operators with the same precedence share an operand, the operators execute from left to right.

Now, let's go back to the previous math expression:

```
outcome = 12.0 + 6.0 / 3.0
```

The value that will be assigned to `outcome` is 14.0 because the division operator has a higher *precedence* than the addition operator. As a result, the division takes place before the addition. The expression can be diagrammed as shown in [Figure 2-9](#).

Figure 2-9 Operator precedence



[Table 2-4](#) shows some other sample expressions with their values.

Table 2-4 Some expressions

Expression	Value
<code>5 + 2 * 4</code>	13
<code>10 / 2 - 3</code>	2.0

<code>8 + 12 * 2 - 4</code>	28
<code>6 - 3 * 2 + 7 - 1</code>	6



Note:

There is an exception to the left-to-right rule. When two `**` operators share an operand, the operators execute right-to-left. For example, the expression `2**3**4` is evaluated as `2**(3**4)`.

Grouping with Parentheses

Parts of a mathematical expression may be grouped with parentheses to force some operations to be performed before others. In the following statement, the variables `a` and `b` are added together, and their sum is divided by 4:

```
result = (a + b) / 4
```

Without the parentheses, however, `b` would be divided by 4 and the result added to `a`. **Table 2-5** shows more expressions and their values.

Table 2-5 More expressions and their values

Expression	Value
<code>(5 + 2) * 4</code>	28
<code>10 / (5 - 3)</code>	5.0
<code>8 + 12 * (6 - 2)</code>	56
<code>(6 - 3) * (2 + 7) / 3</code>	9.0



In the

Spotlight: Calculating an Average

Determining the average of a group of values is a simple calculation: add all of the values then divide the sum by the number of values. Although this is a straightforward calculation, it is easy to make a mistake when writing a program that calculates an average. For example, let's assume that the variables `a`, `b`, and `c` each hold a value and we want to calculate the average of those values. If we are careless, we might write a statement such as the following to perform the calculation:

```
average = a + b + c / 3.0
```

Can you see the error in this statement? When it executes, the division will take place first. The value in `c` will be divided by 3, then the result will be added to `a + b`. That is not the correct way to calculate an average. To correct this error, we need to put parentheses around `a + b + c`, as shown here:

```
average = (a + b + c) / 3.0
```

Let's step through the process of writing a program that calculates an average. Suppose you have taken three tests in your computer science class, and you want to write a program that will display the average of the test scores. Here is the algorithm:

1. *Get the first test score.*
2. *Get the second test score.*
3. *Get the third test score.*
4. *Calculate the average by adding the three test scores and dividing the sum by 3.*
5. *Display the average.*

In steps 1, 2, and 3 we will prompt the user to enter the three test scores. We will store those test scores in the variables `test1`, `test2`, and `test3`. In step 4, we will calculate the average of the three test scores. We will use the following

statement to perform the calculation and store the result in the `average` variable:

```
average = (test1 + test2 + test3) / 3.0
```

Last, in step 5, we display the average. [Program 2-16](#)  shows the program.

Program 2-16

(test_score_average.py)

```
1  # Get three test scores and assign them to the
2  # test1, test2, and test3 variables.
3  test1 = float(input('Enter the first test score:
4  test2 = float(input('Enter the second test score:
5  test3 = float(input('Enter the third test score:
6
7  # Calculate the average of the three scores
8  # and assign the result to the average variable.
9  average = (test1 + test2 + test3) / 3.0
10
11 # Display the average.
```

```
12 print('The average score is', average)
```

Program Output (with input shown in bold)

```
Enter the first test score: 90   
Enter the second test score: 80   
Enter the third test score: 100   
The average score is 90.0
```

The Exponent Operator

In addition to the basic math operators for addition, subtraction, multiplication, and division, Python also provides an exponent operator. Two asterisks written together (`**`) is the exponent operator, and its purpose is to raise a number to a power. For example, the following statement raises the `length` variable to the power of 2 and assigns the result to the `area` variable:

```
area = length**2
```

The following session with the interactive interpreter shows the values of the expressions `4**2`, `5**3`, and `2**10`:

```
>>> 4**2 Enter
```

```
16
```

```
>>> 5**3 Enter
```

```
125
```

```
>>> 2**10 Enter
```

```
1024
```

```
>>>
```

The Remainder Operator

In Python, the `%` symbol is the remainder operator. (This is also known as the *modulus operator*.) The remainder operator performs division, but instead of returning the quotient, it returns the remainder. The following statement assigns 2 to `leftover`:

```
leftover = 17 % 3
```

This statement assigns 2 to `leftover` because 17 divided by 3 is 5 with a remainder of 2. The remainder operator is useful in certain situations. It is commonly used in calculations that convert times or distances, detect odd or even numbers, and perform other specialized operations. For example, [Program 2-17](#)  gets a number of seconds from the user, and it converts that number of seconds to hours,

minutes, and seconds. For example, it would convert 11,730 seconds to 3 hours, 15 minutes, and 30 seconds.

Program 2-17 (*time_converter.py*)

```
1  # Get a number of seconds from the user.
2  total_seconds = float(input('Enter a number of seconds:
3  '))
4  # Get the number of hours.
5  hours = total_seconds // 3600
6
7  # Get the number of remaining minutes.
8  minutes = (total_seconds // 60) % 60
9
10 # Get the number of remaining seconds.
11 seconds = total_seconds % 60
12
13 # Display the results.
14 print('Here is the time in hours, minutes, and seconds:')
15 print('Hours:', hours)
16 print('Minutes:', minutes)
17 print('Seconds:', seconds)
```

Program Output (with input shown in bold)


```
Enter a number of seconds: 11730   
Here is the time in hours, minutes, and seconds:  
Hours: 3.0  
Minutes: 15.0  
Seconds: 30.0
```

Let's take a closer look at the code:

- Line 2 gets a number of seconds from the user, converts the value to a `float`, and assigns it to the `total_seconds` variable.
- Line 5 calculates the number of hours in the specified number of seconds. There are 3600 seconds in an hour, so this statement divides `total_seconds` by 3600. Notice we used the integer division operator (`//`) operator. This is because we want the number of hours with no fractional part.
- Line 8 calculates the number of remaining minutes. This statement first uses the `//` operator to divide `total_seconds` by 60. This gives us the total number of minutes. Then, it uses the `%` operator to divide the total number of minutes by 60 and get the remainder of the division. The result is the number of remaining minutes.
- Line 11 calculates the number of remaining seconds. There are 60 seconds in a minute, so this statement uses the `%` operator to divide the `total_seconds` by 60 and get the remainder of the division. The result is the number of remaining seconds.
- Lines 14 through 17 display the number of hours, minutes, and seconds.

Converting Math Formulas to Programming Statements

You probably remember from algebra class that the expression $2xy$ is understood to mean 2 times x times y . In math, you do not always use an operator for multiplication. Python, as well as other programming languages, requires an operator for any mathematical operation.

Table 2-6  shows some algebraic expressions that perform multiplication and the equivalent programming expressions.

Table 2-6 Algebraic expressions

Algebraic Expression	Operation Being Performed	Programming Expression
$6B$	6 times B	<code>6 * B</code>
$(3)(12)$	3 times 12	<code>3 * 12</code>
$4xy$	4 times x times y	<code>4 * x * y</code>

When converting some algebraic expressions to programming expressions, you may have to insert parentheses that do not appear in the algebraic expression. For example, look at the following formula:

$$x = a + bc$$

To convert this to a programming statement, $a + b$ will have to be enclosed in parentheses:

$$x=(a+b)/c$$

Table 2-7  shows additional algebraic expressions and their Python equivalents.

Table 2-7 Algebraic and programming expressions

Algebraic Expression	Python Statement
$y=3x2$	<code>y = 3 * x / 2</code>
$z=3bc+4$	<code>z = 3 * b * c + 4</code>
$a=x+2b-1$	<code>a = (x + 2) / (b - 1)</code>



In the

Spotlight: Converting a Math Formula to a Programming Statement

Suppose you want to deposit a certain amount of money into a savings account and leave it alone to draw interest for the next 10 years. At the end of 10 years, you would like to have \$10,000 in the account. How much do you need to deposit today to make that happen? You can use the following formula to find out:

$$P=F(1+r)^n$$

The terms in the formula are as follows:

- P is the present value, or the amount that you need to deposit today.
- F is the future value that you want in the account. (In this case, F is \$10,000.)
- r is the annual interest rate.
- n is the number of years that you plan to let the money sit in the account.

It would be convenient to write a computer program to perform the calculation because then we can experiment with different values for the variables. Here is an algorithm that we can use:

1. *Get the desired future value.*
2. *Get the annual interest rate.*
3. *Get the number of years that the money will sit in the account.*
4. *Calculate the amount that will have to be deposited.*
5. *Display the result of the calculation in step 4.*

In steps 1 through 3, we will prompt the user to enter the specified values. We will assign the desired future value to a variable named `future_value`, the annual interest rate to a variable named `rate`, and the number of years to a variable named `years`.

In step 4, we calculate the present value, which is the amount of money that we will have to deposit. We will convert the

formula previously shown to the following statement. The statement stores the result of the calculation in the `present_value` variable.

```
present_value = future_value / (1.0 + rate)**years
```

In step 5, we display the value in the `present_value` variable.

Program 2-18  shows the program.

Program 2-18 (future_value.py)

```
1  # Get the desired future value.
2  future_value = float(input('Enter the desired
future value: '))
3
4  # Get the annual interest rate.
5  rate = float(input('Enter the annual interest rate:
'))
6
7  # Get the number of years that the money will
appreciate.
8  years = int(input('Enter the number of years the
money will grow: '))
9
10 # Calculate the amount needed to deposit.
11 present_value = future_value / (1.0 + rate)**years
```

```
12
13 # Display the amount needed to deposit.
14 print('You will need to deposit this amount:',
present_value)
```

Program Output

```
Enter the desired future value: 10000.0 
Enter the annual interest rate: 0.05 
Enter the number of years the money will grow: 10 
You will need to deposit this amount: 6139.13253541
```



Note:

Unlike the output shown for this program, dollar amounts are usually rounded to two decimal places. Later in this chapter, you will learn how to format numbers so they are rounded to a specified number of decimal places.

Mixed-Type Expressions and Data Type Conversion

When you perform a math operation on two operands, the data type of the result will depend on the data type of the operands. Python follows these rules when evaluating mathematical expressions:

- When an operation is performed on two `int` values, the result will be an `int`.
- When an operation is performed on two `float` values, the result will be a `float`.
- When an operation is performed on an `int` and a `float`, the `int` value will be temporarily converted to a `float` and the result of the operation will be a `float`. (An expression that uses operands of different data types is called a *mixed-type expression*.)

The first two situations are straightforward: operations on `ints` produce `ints`, and operations on `floats` produce `float`s. Let's look at an example of the third situation, which involves mixed-type expressions:

```
my_number = 5 * 2.0
```

When this statement executes, the value 5 will be converted to a `float` (5.0) then multiplied by 2.0. The result, 10.0, will be assigned to `my_number`.

The `int` to `float` conversion that takes place in the previous statement happens implicitly. If you need to explicitly perform a conversion, you can use either the `int()` or `float()` functions. For example, you can use the `int()` function to convert a floating-point value to an integer, as shown in the following code:

```
fvalue = 2.6  
ivalue = int(fvalue)
```

The first statement assigns the value 2.6 to the `fvalue` variable. The second statement passes `fvalue` as an argument to the `int()` function. The `int()` function returns the value 2, which is assigned to the `ivalue` variable. After this code executes, the `fvalue` variable is still assigned the value 2.6, but the `ivalue` variable is assigned the value 2.

As demonstrated in the previous example, the `int()` function converts a floating-point argument to an integer by truncating it. As previously mentioned, that means it throws away the number's fractional part. Here is an example that uses a negative number:

```
fvalue = -2.9  
ivalue = int(fvalue)
```


In the second statement, the value `-2` is returned from the `int()` function. After this code executes, the `fvalue` variable references the value `-2.9`, and the `ivalue` variable references the value `-2`.

You can use the `float()` function to explicitly convert an `int` to a `float`, as shown in the following code:

```
ivalue = 2
fvalue = float(ivalue)
```

After this code executes, the `ivalue` variable references the integer value `2`, and the `fvalue` variable references the floating-point value `2.0`.

Breaking Long Statements into Multiple Lines

Most programming statements are written on one line. If a programming statement is too long, however, you will not be able to view all of it in your editor window without scrolling horizontally. In addition, if you print your program code on paper and one of the statements is too long to fit on one line, it will wrap around to the next line and make the code difficult to read.

Python allows you to break a statement into multiple lines by using the *line continuation character*, which is a backslash (`\`). You simply type the backslash character at the point you want to break the statement, then press the Enter key.

For example, here is a statement that performs a mathematical calculation and has been broken up to fit on two lines:

```
result = var1 * 2 + var2 * 3 + \  
        var3 * 4 + var4 * 5
```

The line continuation character that appears at the end of the first line tells the interpreter that the statement is continued on the next line.

Python also allows you to break any part of a statement that is enclosed in parentheses into multiple lines without using the line continuation character. For example, look at the following statement:

```
print("Monday's sales are", monday,  
      "and Tuesday's sales are", tuesday,  
      "and Wednesday's sales are", wednesday)
```

The following code shows another example:

```
total = (value1 + value2 +
```

```
value3 + value4 +
```

```
value5 + value6)
```



Checkpoint

2.19 Complete the following table by writing the value of each expression in the Value column:

Expression	Value
<code>6 + 3 * 5</code>	_____
<code>12 / 2 - 4</code>	_____
<code>9 + 14 * 2 - 6</code>	_____
<code>(6 + 2) * 3</code>	_____
<code>14 / (11 - 4)</code>	_____
<code>9 + 12 * (8 - 3)</code>	_____

2.20 What value will be assigned to `result` after the following statement executes?

```
result = 9 // 2
```

2.21 What value will be assigned to `result` after the following statement executes?

```
result = 9 % 2
```

2.8 More About Data Output

So far, we have discussed only basic ways to display data. Eventually, you will want to exercise more control over the way data appear on the screen. In this section, you will learn more details about the Python `print` function, and you'll see techniques for formatting output in specific ways.

Suppressing the `print` Function's Ending Newline

The `print` function normally displays a line of output. For example, the following three statements will produce three lines of output:

```
print('One')  
print('Two')  
print('Three')
```

Each of the statements shown here displays a string and then prints a *newline character*. You do not see the newline character, but when it is displayed, it causes the output to advance to the next line. (You can

think of the newline character as a special command that causes the computer to start a new line of output.)

If you do not want the `print` function to start a new line of output when it finishes displaying its output, you can pass the special argument `end=' '` to the function, as shown in the following code:

```
print('One', end=' ')\nprint('Two', end=' ')\nprint('Three')
```

Notice in the first two statements, the argument `end=' '` is passed to the `print` function. This specifies that the `print` function should print a space instead of a newline character at the end of its output. Here is the output of these statements:

```
One Two Three
```

Sometimes, you might not want the `print` function to print anything at the end of its output, not even a space. If that is the case, you can pass the argument `end=''` to the `print` function, as shown in the following code:

```
print('One', end='')
```

```
print('Two', end='')  
print('Three')
```

Notice in the argument `end=''` there is no space between the quote marks. This specifies that the `print` function should print nothing at the end of its output. Here is the output of these statements:

```
One Two Three
```

Specifying an Item Separator

When multiple arguments are passed to the `print` function, they are automatically separated by a space when they are displayed on the screen. Here is an example, demonstrated in interactive mode:

```
>>> print('One', 'Two', 'Three')   
One Two Three  
>>>
```

If you do not want a space printed between the items, you can pass the argument `sep=''` to the `print` function, as shown here:

```
>>> print('One', 'Two', 'Three', sep='') Enter
```

```
OneTwoThree
```

```
>>>
```

You can also use this special argument to specify a character other than the space to separate multiple items. Here is an example:

```
>>> print('One', 'Two', 'Three', sep='*') Enter
```

```
One*Two*Three
```

```
>>>
```

Notice in this example, we passed the argument `sep='*'` to the `print` function. This specifies that the printed items should be separated with the `*` character. Here is another example:

```
>>> print('One', 'Two', 'Three', sep='~~~') Enter
```

```
One~~~Two~~~Three
```

```
>>>
```

Escape Characters

An *escape character* is a special character that is preceded with a backslash (`\`), appearing inside a string literal. When a string literal that contains escape characters is printed, the escape characters are treated as special commands that are embedded in the string.

For example, `\n` is the newline escape character. When the `\n` escape character is printed, it isn't displayed on the screen. Instead, it causes output to advance to the next line. For example, look at the following statement:

```
print('One\nTwo\nThree')
```

When this statement executes, it displays

```
One
Two
Three
```

Python recognizes several escape characters, some of which are listed in [Table 2-8](#) .

Table 2-8 Some of Python's escape characters

Escape Character	Effect
<code>\n</code>	Causes output to be advanced to the next line.

<code>\t</code>	Causes output to skip over to the next horizontal tab position.
<code>\'</code>	Causes a single quote mark to be printed.
<code>\"</code>	Causes a double quote mark to be printed.
<code>\\</code>	Causes a backslash character to be printed.

The `\t` escape character advances the output to the next horizontal tab position. (A tab position normally appears after every eighth character.) The following statements are illustrative:

```
print('Mon\tTues\tWed')
print('Thur\tFri\tSat')
```

This statement prints `Monday`, then advances the output to the next tab position, then prints `Tuesday`, then advances the output to the next tab position, then prints `Wednesday`. The output will look like this:

```
Mon    Tues    Wed
Thur   Fri     Sat
```

You can use the `\'` and `\"` escape characters to display quotation marks. The following statements are illustrative:

```
print("Your assignment is to read \"Hamlet\" by tomorrow.")  
print('I\'m ready to begin.')
```

These statements display the following:

```
Your assignment is to read "Hamlet" by tomorrow.  
I'm ready to begin.
```

You can use the `\\` escape character to display a backslash, as shown in the following:

```
print('The path is C:\\temp\\data.')
```

This statement will display

```
The path is C:\temp\data.
```

Displaying Multiple Items with the + Operator

Earlier in this chapter, you saw that the + operator is used to add two numbers. When the + operator is used with two strings, however, it performs *string concatenation*. This means that it appends one string to another. For example, look at the following statement:

```
print('This is ' + 'one string.')
```

This statement will print

```
This is one string.
```

String concatenation can be useful for breaking up a string literal so a lengthy call to the `print` function can span multiple lines. Here is an example:

```
print('Enter the amount of ' +  
      'sales for each day and ' +  
      'press Enter.')
```

This statement will display the following:

```
Enter the amount of sales for each day and press Enter.
```

Formatting Numbers

You might not always be happy with the way that numbers, especially floating-point numbers, are displayed on the screen. When a floating-point number is displayed by the `print` function, it can appear with up to 12 significant digits. This is shown in the output of [Program 2-19](#).

Program 2-19 `(no_formatting.py)`

```
1 # This program demonstrates how a floating-point
2 # number is displayed with no formatting.
3 amount_due = 5000.0
4 monthly_payment = amount_due / 12.0
5 print('The monthly payment is', monthly_payment)
```

Program Output

```
The monthly payment is 416.666666667
```

Because this program displays a dollar amount, it would be nice to see that amount rounded to two decimal places. Fortunately, Python gives us a way to do just that, and more, with the built-in `format` function.

When you call the built-in `format` function, you pass two arguments to the function: a numeric value and a format specifier. The *format specifier* is a string that contains special characters specifying how the numeric value should be formatted. Let's look at an example:

```
format(12345.6789, '.2f')
```

The first argument, which is the floating-point number 12345.6789, is the number that we want to format. The second argument, which is the string `'.2f'`, is the format specifier. Here is the meaning of its contents:

- The `.2` specifies the precision. It indicates that we want to round the number to two decimal places.
- The `f` specifies that the data type of the number we are formatting is a floating-point number.

The `format` function returns a string containing the formatted number. The following interactive mode session demonstrates how you use the `format` function along with the `print` function to display a formatted number:

```
>>> print(format(12345.6789, '.2f')) Enter
```

```
12345.68
```

```
>>>
```

Notice the number is rounded to two decimal places. The following example shows the same number, rounded to one decimal place:

```
>>> print(format(12345.6789, '.1f'))   
12345.7  
>>>
```

Here is another example:

```
>>> print('The number is', format(1.234567, '.2f'))   
The number is 1.23  
>>>
```

Program 2-20  shows how we can modify **Program 2-19**  so it formats its output using this technique.

Program 2-20 *(formatting.py)*

```
1 # This program demonstrates how a floating-point  
2 # number can be formatted.  
3 amount_due = 5000.0  
4 monthly_payment = amount_due / 12  
5 print('The monthly payment is',  
6       format(monthly_payment, '.2f'))
```

Program Output

```
The monthly payment is 416.67
```

Formatting in Scientific Notation

If you prefer to display floating-point numbers in scientific notation, you can use the letter `e` or the letter `E` instead of `f`. Here are some examples:

```
>>> print(format(12345.6789, 'e'))   
1.234568e+04  
>>> print(format(12345.6789, '.2e'))   
1.23e+04  
>>>
```

The first statement simply formats the number in scientific notation. The number is displayed with the letter `e` indicating the exponent. (If you use uppercase `E` in the format specifier, the result will contain an uppercase `E` indicating the exponent.) The second statement additionally specifies a precision of two decimal places.

Inserting Comma Separators

If you want the number to be formatted with comma separators, you can insert a comma into the format specifier, as shown here:

```
>>> print(format(12345.6789, ',.2f'))   
12,345.68  
>>>
```

Here is an example that formats an even larger number:

```
>>> print(format(123456789.456, ',.2f'))   
123,456,789.46  
>>>
```

Notice in the format specifier, the comma is written before (to the left of) the precision designator. Here is an example that specifies the comma separator, but does not specify precision:

```
>>> print(format(12345.6789, ',f'))   
12,345.678900  
>>>
```

Program 2-21  demonstrates how the comma separator and a precision of two decimal places can be used to format larger numbers as currency amounts.

Program 2-21 (*dollar_display.py*)

```
1 # This program demonstrates how a floating-point
2 # number can be displayed as currency.
3 monthly_pay = 5000.0
4 annual_pay = monthly_pay * 12
5 print('Your annual pay is $',
6       format(annual_pay, ',.2f'),
7       sep='')
```

Program Output

```
Your annual pay is $60,000.00
```

Notice in line 7, we passed the argument `sep=''` to the `print` function. As we mentioned earlier, this specifies that no space should be printed between the items that are being displayed. If we did not pass this argument, a space would be printed after the \$ sign.

Specifying a Minimum Field Width

The format specifier can also include a minimum field width, which is the minimum number of spaces that should be used to display the value. The following example prints a number in a field that is 12 spaces wide:

```
>>> print('The number is', format(12345.6789, '12,.2f')) Enter  
The number is 12,345.68  
>>>
```

In this example, the 12 that appears in the format specifier indicates that the number should be displayed in a field that is a minimum of 12 spaces wide. In this case, the number that is displayed is shorter than the field that it is displayed in. The number `12,345.68` uses only 9 spaces on the screen, but it is displayed in a field that is 12 spaces wide. When this is the case, the number is right justified in the field. If a value is too large to fit in the specified field width, the field is automatically enlarged to accommodate it.

Note in the previous example, the field width designator is written before (to the left of) the comma separator. Here is an example that specifies field width and precision, but does not use comma separators:

```
>>> print('The number is', format(12345.6789, '12.2f')) Enter  
The number is      12345.68  
>>>
```

Field widths can help when you need to print numbers aligned in columns. For example, look at [Program 2-22](#). Each of the variables is displayed in a field that is seven spaces wide.

Program 2-22 (*columns.py*)

```
1  # This program displays the following
2  # floating-point numbers in a column
3  # with their decimal points aligned.
4  num1 = 127.899
5  num2 = 3465.148
6  num3 = 3.776
7  num4 = 264.821
8  num5 = 88.081
9  num6 = 799.999
10
11 # Display each number in a field of 7 spaces
12 # with 2 decimal places.
13 print(format(num1, '7.2f'))
14 print(format(num2, '7.2f'))
15 print(format(num3, '7.2f'))
16 print(format(num4, '7.2f'))
17 print(format(num5, '7.2f'))
18 print(format(num6, '7.2f'))
```

Program Output

```
127.90
3465.15
3.78
264.82
88.08
800.00
```

Formatting a Floating-Point Number as a Percentage

Instead of using `f` as the type designator, you can use the `%` symbol to format a floating-point number as a percentage. The `%` symbol causes the number to be multiplied by 100 and displayed with a `%` sign following it. Here is an example:

```
>>> print(format(0.5, '%')) Enter
50.000000%
>>>
```

Here is an example that specifies 0 as the precision:

```
>>> print(format(0.5, '.0%')) Enter
```

```
50%
```

```
>>>
```

Formatting Integers

All the previous examples demonstrated how to format floating-point numbers. You can also use the `format` function to format integers.

There are two differences to keep in mind when writing a format specifier that will be used to format an integer:

- You use `d` as the type designator.
- You cannot specify precision.

Let's look at some examples in the interactive interpreter. In the following session, the number 123456 is printed with no special formatting:

```
>>> print(format(123456, 'd')) Enter
```

```
123456
```

```
>>>
```

In the following session, the number 123456 is printed with a comma separator:

```
>>> print(format(123456, ',d')) Enter
```

```
123,456
```

```
>>>
```

In the following session, the number 123456 is printed in a field that is 10 spaces wide:

```
>>> print(format(123456, '10d')) Enter
```

```
123456
```

```
>>>
```

In the following session, the number 123456 is printed with a comma separator in a field that is 10 spaces wide:

```
>>> print(format(123456, '10,d'))
```

```
123,456
```

```
>>>
```



Checkpoint

2.22 How do you suppress the `print` function's ending newline?

2.23 How can you change the character that is automatically displayed between multiple items that are passed to the `print`

function?

2.24 What is the `'\n'` escape character?

2.25 What does the `+` operator do when it is used with two strings?

2.26 What does the statement `print(format(65.4321, '.2f'))` display?

2.27 What does the statement `print(format(987654.129, ',.2f'))` display?

2.9 Named Constants

Concept:

A named constant is a name that represents a value that cannot be changed during the program's execution.

Imagine, for a moment, that you are a programmer working for a bank. You are updating an existing program that calculates data pertaining to loans, and you see the following line of code:

```
amount = balance * 0.069
```

Because someone else wrote the program, you aren't sure what the number 0.069 is. It appears to be an interest rate, but it could be a number that is used to calculate some sort of fee. You simply can't determine the purpose of the number 0.069 by reading this line of code. This is an example of a magic number. A *magic number* is an unexplained value that appears in a program's code.

Magic numbers can be problematic, for a number of reasons. First, as illustrated in our example, it can be difficult for someone reading the code to determine the purpose of the number. Second, if the magic number is used in multiple places in the program, it can take painstaking effort to change the number in each location, should the need arise. Third, you take the risk of making a typographical mistake each time you type the magic number in the program's code. For example, suppose you intend to type 0.069, but you accidentally type .0069. This mistake will cause mathematical errors that can be difficult to find.

These problems can be addressed by using named constants to represent magic numbers. A *named constant* is a name that represents a value that does not change during the program's execution. The following is an example of how we will declare named constants in our code:

```
INTEREST_RATE = 0.069
```

This creates a named constant named `INTEREST_RATE` that is assigned the value 0.069. Notice the named constant is written in all uppercase letters. This is a standard practice in most programming languages because it makes named constants easily distinguishable from regular variables.

One advantage of using named constants is that they make programs more self-explanatory. The following statement:

```
amount = balance * 0.069
```

can be changed to read

```
amount = balance * INTEREST_RATE
```

A new programmer can read the second statement and know what is happening. It is evident that balance is being multiplied by the interest rate.

Another advantage to using named constants is that widespread changes can easily be made to the program. Let's say the interest rate appears in a dozen different statements throughout the program. When the rate changes, the initialization value in the declaration of the named constant is the only value that needs to be modified. If the rate increases to 7.2 percent, the declaration can be changed to:

```
INTEREST_RATE = 0.072
```

The new value of 0.072 will then be used in each statement that uses the `INTEREST_RATE` constant.

Another advantage to using named constants is that they help to prevent the typographical errors that are common when using magic

numbers. For example, if you accidentally type `.0069` instead of `0.069` in a math statement, the program will calculate the wrong value. However, if you misspell `INTEREST_RATE`, the Python interpreter will display a message indicating that the name is not defined.



Checkpoint

2.28 What are three advantages of using named constants?

2.29 Write a Python statement that defines a named constant for a 10 percent discount.

2.10 Introduction to Turtle Graphics

Concept:

Turtle graphics is an interesting and easy way to learn basic programming concepts. The Python turtle graphics system simulates a “turtle” that obeys commands to draw simple graphics.

In the late 1960s, MIT professor Seymour Papert used a robotic “turtle” to teach programming. The turtle was tethered to a computer where a student could enter commands, causing the turtle to move. The turtle also had a pen that could be raised and lowered, so it could be placed on a sheet of paper and programmed to draw images. Python has a *turtle graphics* system that simulates a robotic turtle. The system displays a small cursor (the turtle) on the screen. You can use Python statements to move the turtle around the screen, drawing lines and shapes.

The first step in using Python’s turtle graphics system is to write the following statement:

```
import turtle
```



Introduction to Turtle Graphics

This statement is necessary because the turtle graphics system is not built into the Python interpreter. Instead, it is stored in a file known as the *turtle module*. The `import turtle` statement loads the turtle module into memory so the Python interpreter can use it.

If you are writing a Python program that uses turtle graphics, you will write the `import` statement at the top of the program. If you want to experiment with turtle graphics in interactive mode, you can type the statement into the Python shell, as shown here:

```
>>> import turtle
```

```
>>>
```

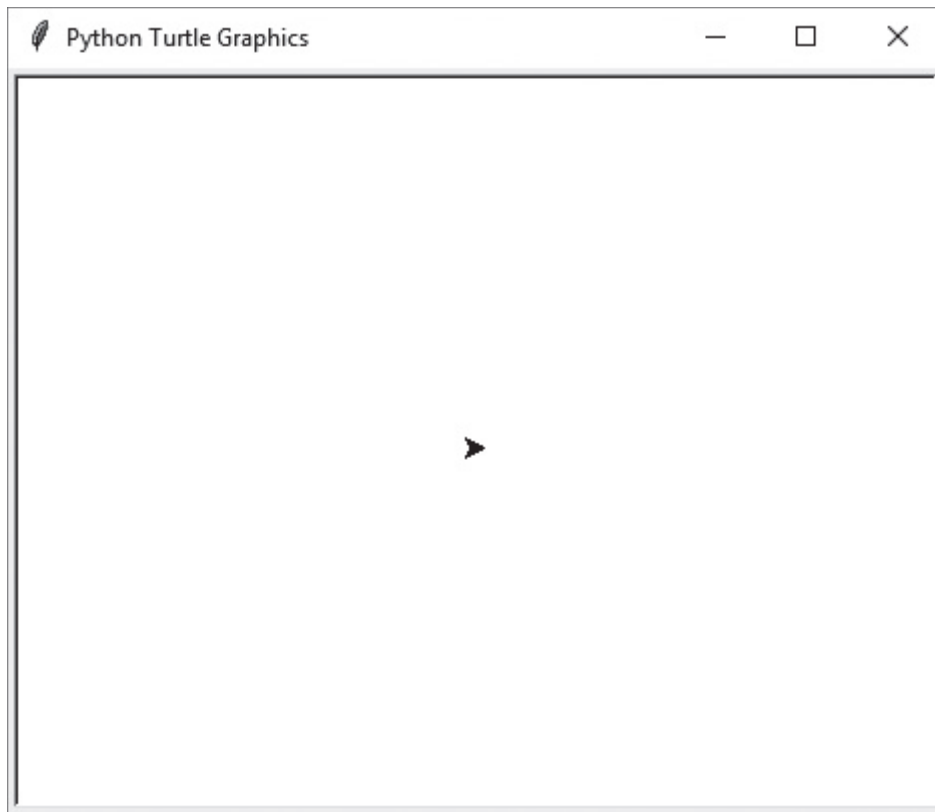
Drawing Lines with the Turtle

The Python turtle is initially positioned in the center of a graphics window that serves as its canvas. In interactive mode, you can enter the `turtle.showturtle()` command to display the turtle in its window. Here is an example session that imports the `turtle` module, and then shows the turtle:

```
>>> import turtle
>>> turtle.showturtle()
```

This causes a graphics window similar to the one shown in **Figure 2-10**  to appear. Notice the turtle doesn't look like a turtle at all. Instead, it looks like an arrowhead (➤). This is important, because it points in the direction that the turtle is currently facing. If we instruct the turtle to move forward, it will move in the direction that the arrowhead is pointing. Let's try it out. You can use the `turtle.forward(n)` command to move the turtle forward `n` pixels. (Just type the desired number of pixels in place of `n`.) For example, the command `turtle.forward(200)` moves the turtle forward 200 pixels. Here is an example of a complete session in the Python shell:

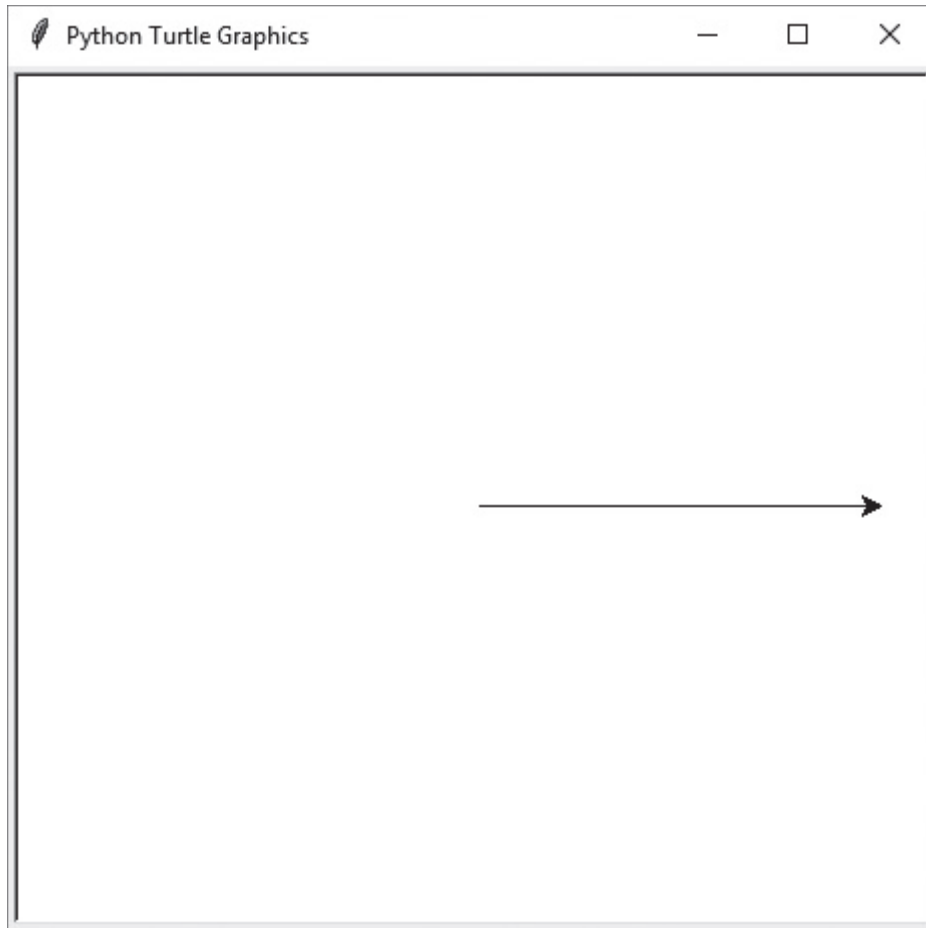
Figure 2-10 The turtle's graphics window



```
>>> import turtle
>>> turtle.forward(200)
>>>
```

Figure 2-11  shows the output of this interactive session. Notice a line was drawn by turtle as it moved forward.

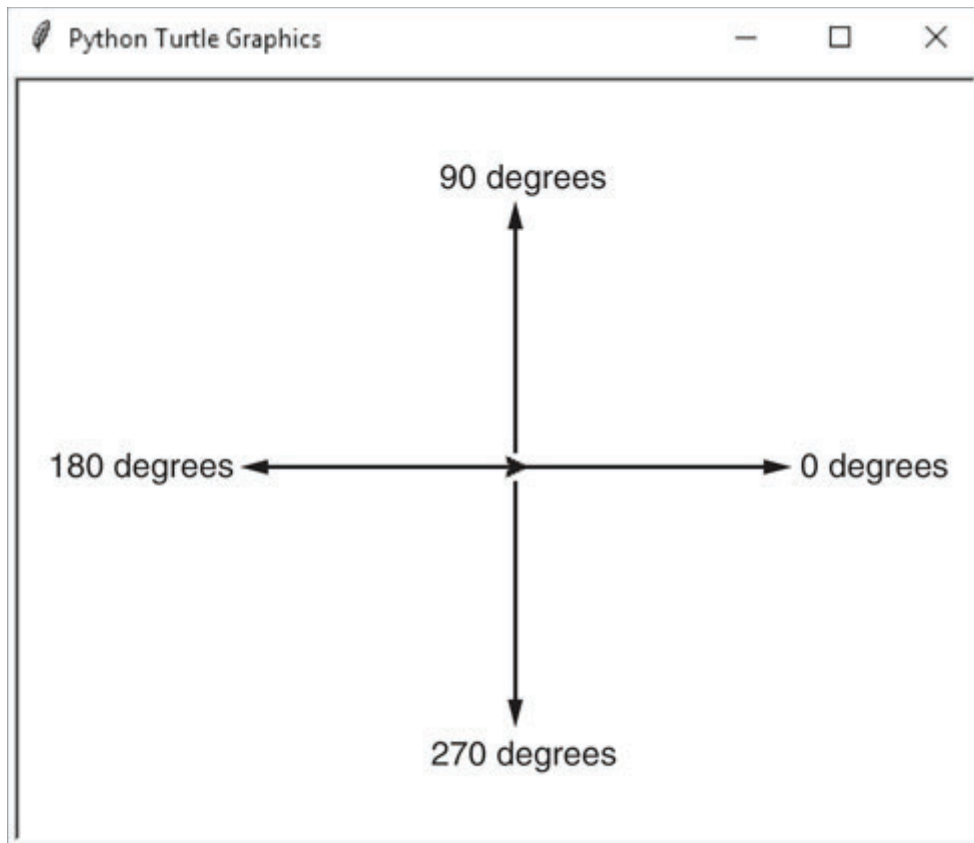
Figure 2-11 The turtle moving forward 200 pixels



Turning the Turtle

When the turtle first appears, its default heading is 0 degrees (East). This is shown in [Figure 2-12](#).

Figure 2-12 The turtle's heading

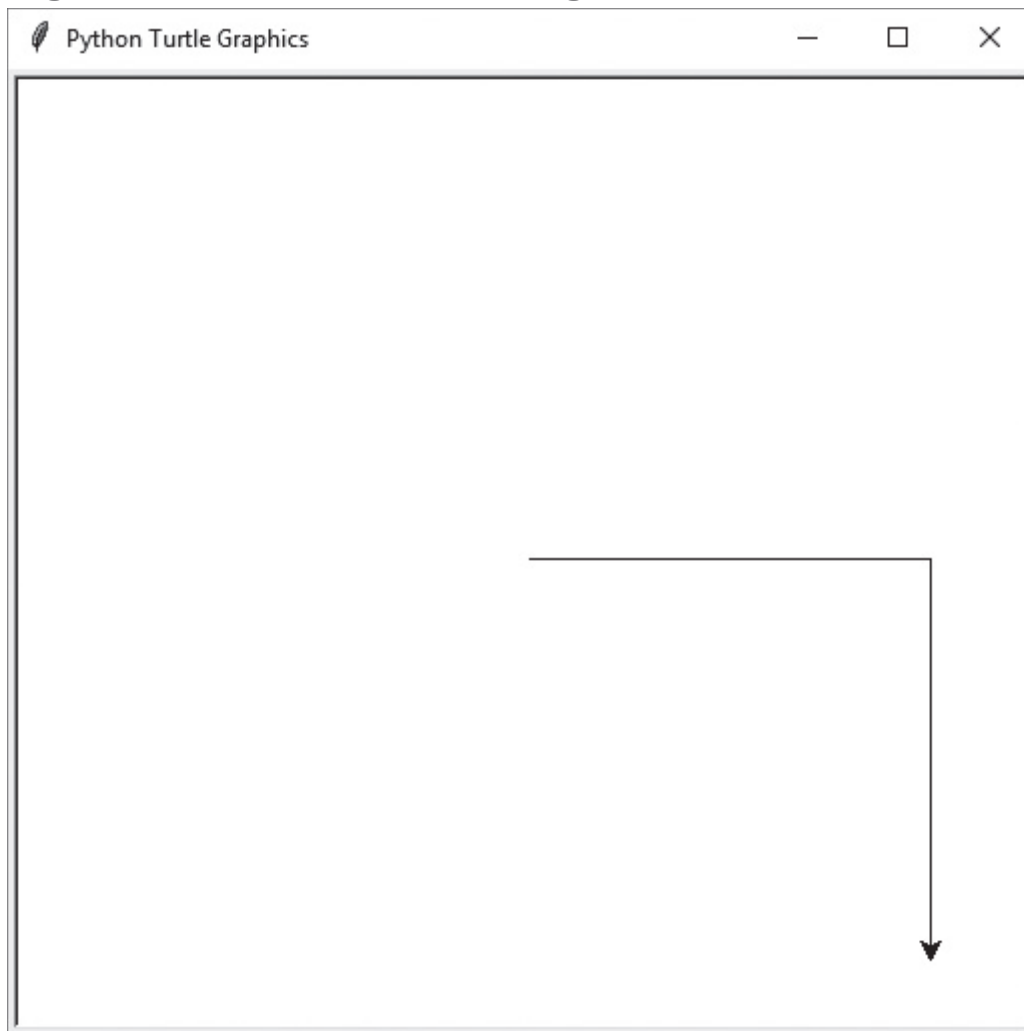


You can turn the turtle so it faces a different direction by using either the `turtle.right(angle)` command, or the `turtle.left(angle)` command. The `turtle.right(angle)` command turns the turtle right by *angle* degrees, and the `turtle.left(angle)` command turns the turtle left by *angle* degrees. Here is an example session that uses the `turtle.right(angle)` command:

```
>>> import turtle
>>> turtle.forward(200)
>>> turtle.right(90)
>>> turtle.forward(200)
>>>
```

This session first moves the turtle forward 200 pixels. Then it turns the turtle right by 90 degrees (the turtle will be pointing down). Then it moves the turtle forward by 200 pixels. The session's output is shown in [Figure 2-13](#) .

Figure 2-13 The turtle turns right

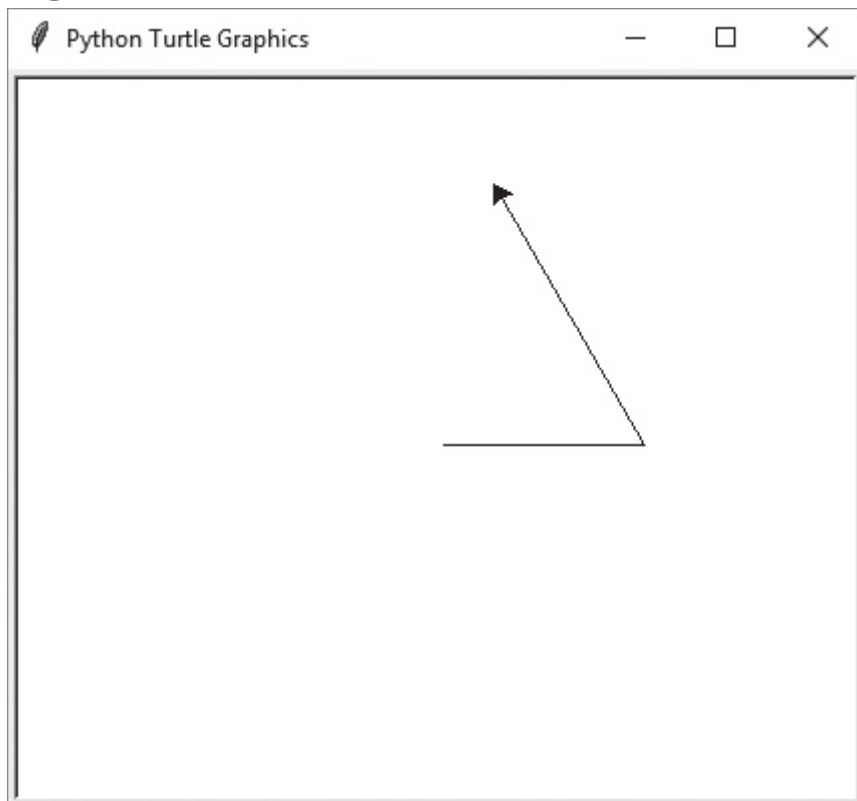


Here is an example session that uses the `turtle.left(angle)` command:

```
>>> import turtle
>>> turtle.forward(100)
>>> turtle.left(120)
>>> turtle.forward(150)
>>>
```

This session first moves the turtle forward 100 pixels. Then it turns the turtle left by 120 degrees (the turtle will be pointing in a Northwestern direction). Then it moves the turtle forward by 150 pixels. The session's output is shown in [Figure 2-14](#).

Figure 2-14 The turtle turns left

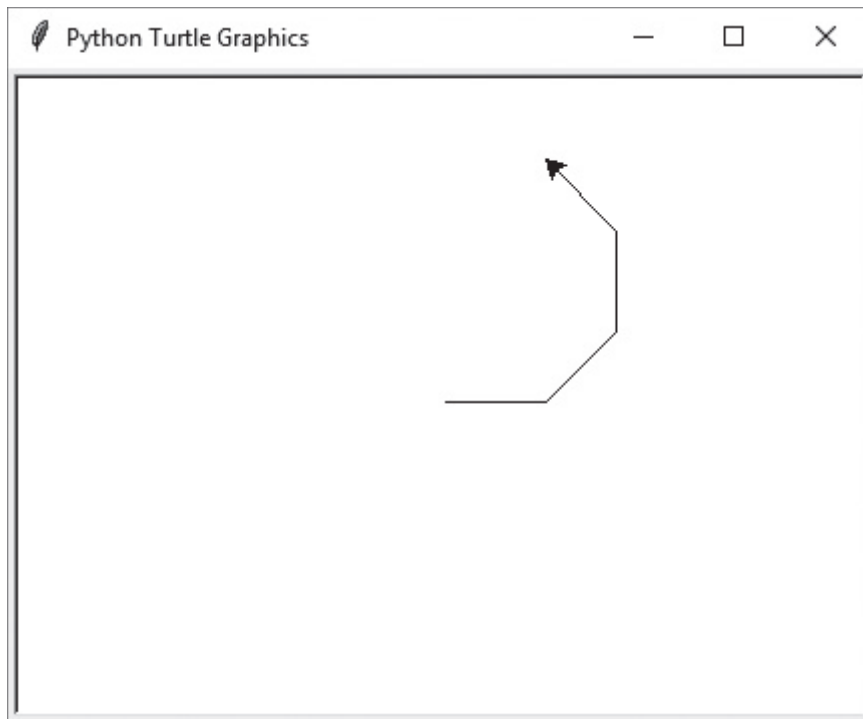


Keep in mind that the `turtle.right` and `turtle.left` commands turn the turtle *by* a specified angle. For example, the turtle's current heading is 90 degrees (due north). If you enter the command `turtle.left(20)`, then the turtle will be turned left by 20 degrees. This means the turtle's heading will be 110 degrees. For another example, look at the following interactive session:

```
>>> import turtle
>>> turtle.forward(50)
>>> turtle.left(45)
>>> turtle.forward(50)
>>> turtle.left(45)
>>> turtle.forward(50)
>>> turtle.left(45)
>>> turtle.forward(50)
>>>
```

Figure 2-15  shows the session's output. At the beginning of this session, the turtle's heading is 0 degrees. In the third line, the turtle is turned left by 45 degrees. In the fifth line, the turtle is turned left again by an additional 45 degrees. In the seventh line, the turtle is once again turned left by 45 degrees. After all of these 45 degree turns, the turtle's heading will finally be 135 degrees.

Figure 2-15 Turning the turtle by 45 degree angles



Setting the Turtle's Heading to a Specific Angle

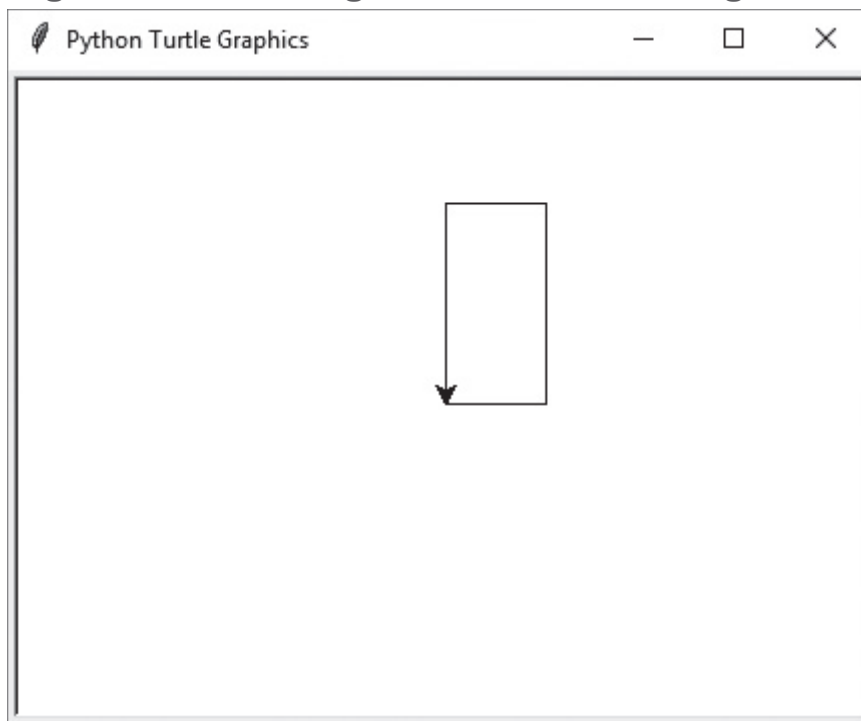
You can use the `turtle.setheading(angle)` command to set the turtle's heading to a specific angle. Simply specify the desired angle as the *angle* argument. The following interactive session shows an example:

```
>>> import turtle
>>> turtle.forward(50)
>>> turtle.setheading(90)
>>> turtle.forward(100)
>>> turtle.setheading(180)
```

```
>>> turtle.forward(50)
>>> turtle.setheading(270)
>>> turtle.forward(100)
>>>
```

As usual, the turtle's initial heading is 0 degrees. In the third line, the turtle's heading is set to 90 degrees. Then, in the fifth line, the turtle's heading is set to 180 degrees. Then, in the seventh line, the turtle's heading is set to 270 degrees. The session's output is shown in [Figure 2-16](#) .

Figure 2-16 Setting the turtle's heading



Getting the Turtle's Current

Heading

In an interactive session, you can use the `turtle.heading()` command to display the turtle's current heading. Here is an example:

```
>>> import turtle
>>> turtle.heading()
0.0
>>> turtle.setheading(180)
>>> turtle.heading()
180.0
>>>
```

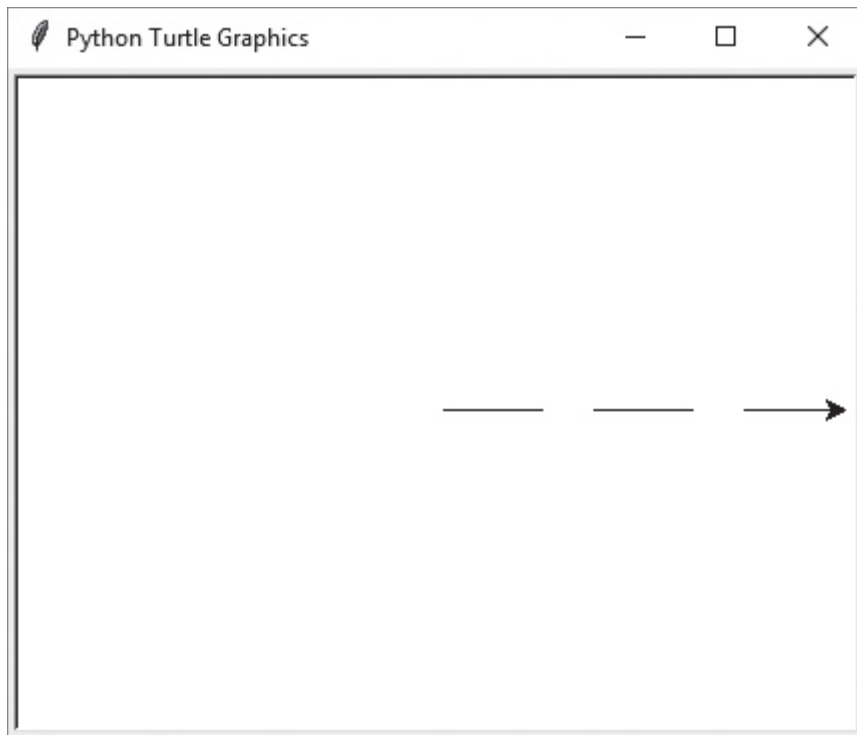
Moving the Pen Up and Down

The original robotic turtle sat on a large sheet of paper, and had a pen that could be raised and lowered. When the pen was down, it was in contact with the paper and would draw a line as the turtle moved. When the pen was up, it was not touching the paper, so the turtle could move without drawing a line.

In Python, you can use the `turtle.penup()` command to raise the pen, and the `turtle.pendown()` command to lower the pen. When the pen is up, you can move the turtle without drawing a line. When the pen is

down, the turtle leaves a line when it is moved. (By default, the pen is down.) The following session shows an example. The session's output is shown in **Figure 2-17** .

Figure 2-17 Raising and lowering the pen



```
>>> import turtle
>>> turtle.forward(50)
>>> turtle.penup()
>>> turtle.forward(25)
>>> turtle.pendown()
>>> turtle.forward(50)
>>> turtle.penup()
>>> turtle.forward(25)
>>> turtle.pendown()
```

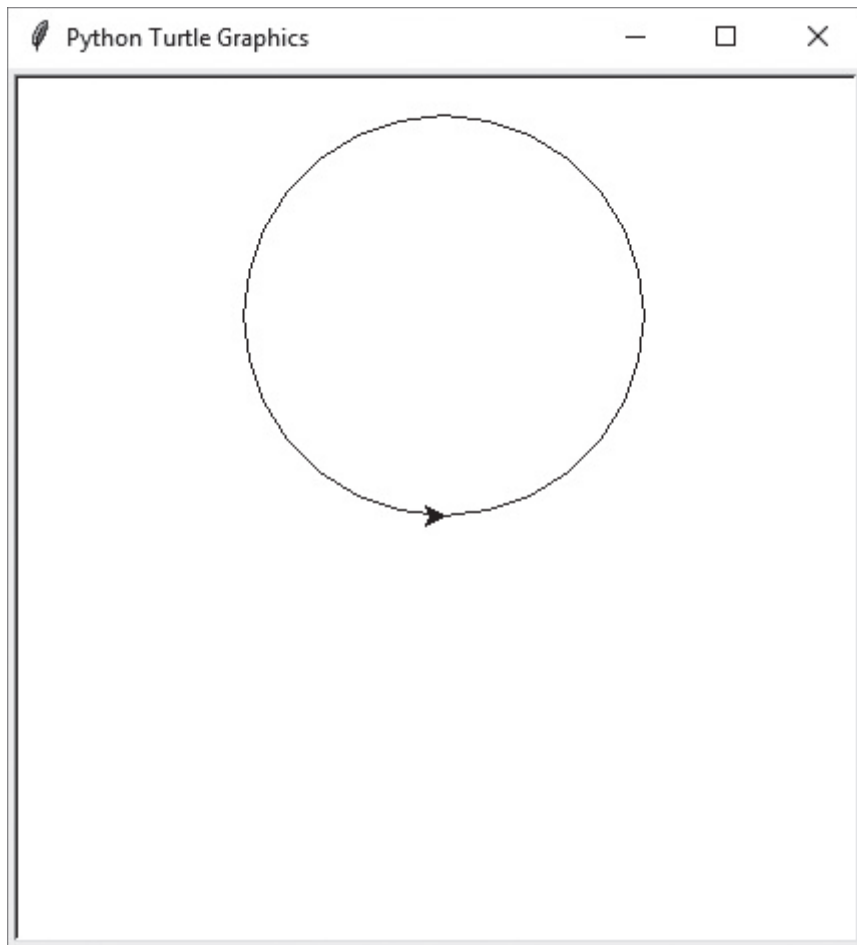
```
>>> turtle.forward(50)
```

```
>>>
```

Drawing Circles and Dots

You can use the `turtle.circle(radius)` command to make the turtle draw a circle with a radius of `radius` pixels. For example, the command `turtle.circle(100)` causes the turtle to draw a circle with a radius of 100 pixels. The following interactive session displays the output shown in **Figure 2-18** .

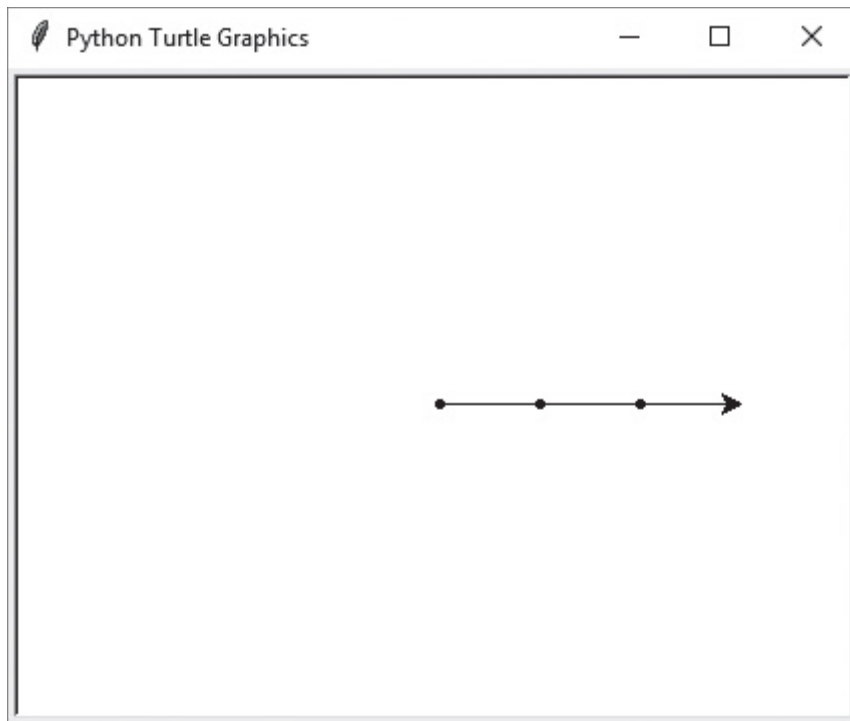
Figure 2-18 A circle



```
>>> import turtle
>>> turtle.circle(100)
>>>
```

You can use the `turtle.dot()` command to make the turtle draw a simple dot. For example, the following interactive session produces the output shown in [Figure 2-19](#) :

Figure 2-19 Drawing dots



```
>>> import turtle
>>> turtle.dot()
>>> turtle.forward(50)
>>> turtle.dot()
>>> turtle.forward(50)
>>> turtle.dot()
>>> turtle.forward(50)
>>>
```

Changing the Pen Size

You can use the `turtle.pensize(width)` command to change the width of the turtle's pen, in pixels. The `width` argument is an integer specifying the pen's width. For example, the following interactive session sets the pen's width to 5 pixels, then draws a circle:

```
>>> import turtle
>>> turtle.pensize(5)
>>> turtle.circle(100)
>>>
```

Changing the Drawing Color

You can use the `turtle.pencolor(color)` command to change the turtle's drawing color. The `color` argument is the name of a color, as a string. For example, the following interactive session changes the drawing color to red, then draws a circle:

```
>>> import turtle
>>> turtle.pencolor('red')
>>> turtle.circle(100)
>>>
```

There are numerous predefined color names that you can use with the `turtle.pencolor` command, and [Appendix D](#) shows the complete list. Some of the more common colors are `'red'`, `'green'`, `'blue'`, `'yellow'`, and `'cyan'`.

Changing the Background Color

You can use the `turtle.bgcolor(color)` command to change the background color of the turtle's graphics window. The `color` argument is the name of a color, as a string. For example, the following interactive session changes the background color to gray, changes the drawing color to red, then draws a circle:

```
>>> import turtle
>>> turtle.bgcolor('gray')
>>> turtle.pencolor('red')
>>> turtle.circle(100)
>>>
```

As previously mentioned, there are numerous predefined color names, and [Appendix D](#) shows the complete list.

Resetting the Screen

There are three commands that you can use to reset the turtle's graphics window: `turtle.reset()`, `turtle.clear()`, and `turtle.clearscreen()`. Here is a summary of the commands:

- The `turtle.reset()` command erases all drawings that currently appear in the graphics window, resets the drawing color to black, and resets the turtle to its original position in the center of the screen. This command does not reset the graphics window's background color.
- The `turtle.clear()` command simply erases all drawings that currently appear in the graphics window. It does not change the turtle's position, the drawing color, or the graphics window's background color.
- The `turtle.clearscreen()` command erases all drawings that currently appear in the graphics window, resets the drawing color to black, resets the graphics window's background color to white, and resets the turtle to its original position in the center of the graphics window.

Specifying the Size of the Graphics Window

You can use the `turtle.setup(width, height)` command to specify a size for the graphics window. The `width` and `height` arguments are the width and height, in pixels. For example, the following interactive

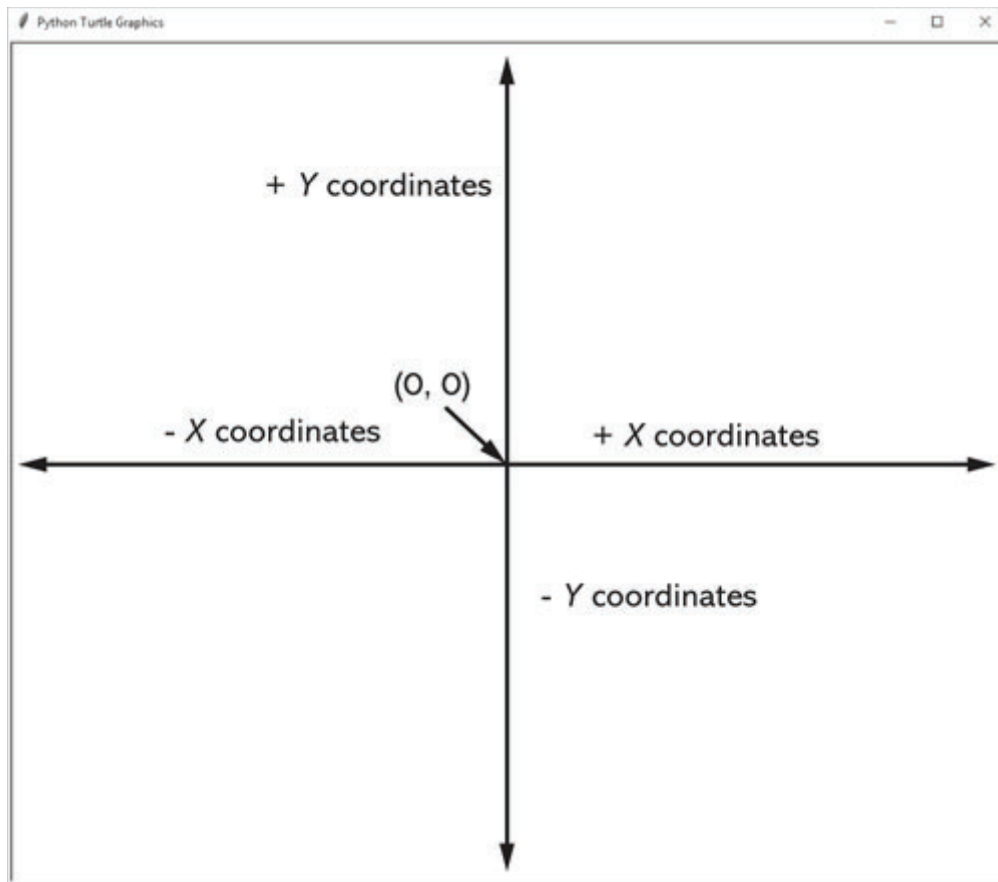
session creates a graphics window that is 640 pixels wide and 480 pixels high:

```
>>> import turtle
>>> turtle.setup(640, 480)
>>>
```

Moving the Turtle to a Specific Location

A Cartesian coordinate system is used to identify the position of each pixel in the turtle's graphics window, as illustrated in [Figure 2-20](#) . Each pixel has an *X* coordinate and a *Y* coordinate. The *X* coordinate identifies the pixel's horizontal position, and the *Y* coordinate identifies its vertical position. Here are the important things to know:

Figure 2-20 Cartesian coordinate system

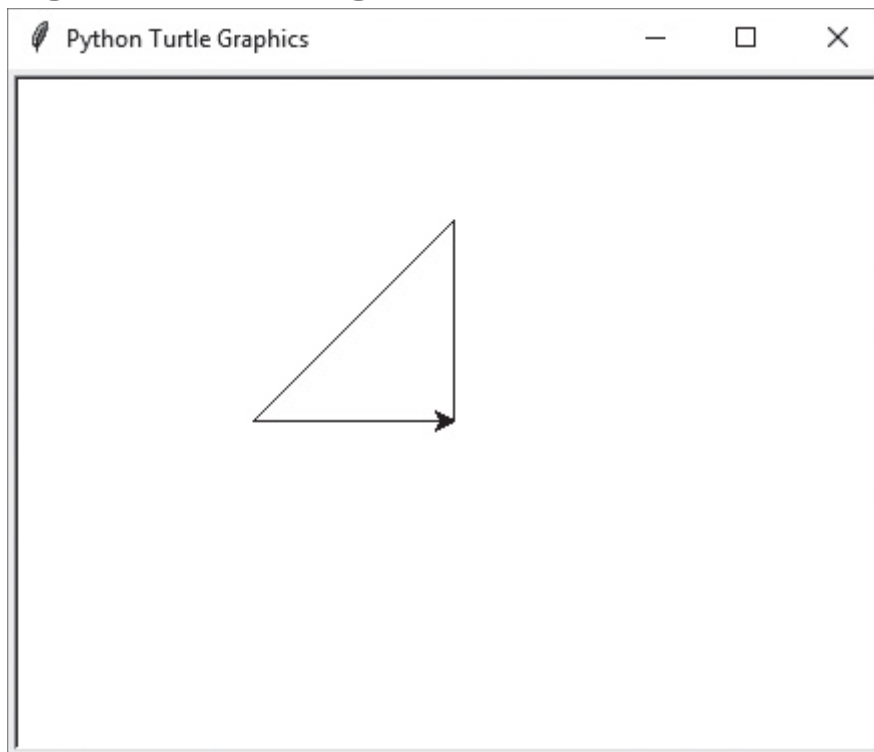


- The pixel in the center of the graphics window is at the position (0, 0), which means that its X coordinate is 0 and its Y coordinate is 0.
- The X coordinates increase in value as we move toward the right side of the window, and they decrease in value as we move toward the left side of the window.
- The Y coordinates increase in value as we move toward the top of the window, and decrease in value as we move toward the bottom of the window.
- Pixels that are located to the right of the center point have positive X coordinates, and pixels that are located to the left of the center point have negative X coordinates.
- Pixels that are located above the center point have positive Y coordinates, and pixels that are located below the center point

have negative *Y* coordinates.

You can use the `turtle.goto(x, y)` command to move the turtle from its current location to a specific position in the graphics window. The `x` and `y` arguments are the coordinates of the position to which to move the turtle. If the turtle's pen is down, a line will be drawn as the turtle moves. For example, the following interactive session draws the lines shown in **Figure 2-21** :

Figure 2-21 Moving the turtle



```
>>> import turtle
>>> turtle.goto(0, 100)
>>> turtle.goto(-100, 0)
>>> turtle.goto(0, 0)
```

```
>>>
```

Getting the Turtle's Current Position

In an interactive session, you can use the `turtle.pos()` command to display the turtle's current position. Here is an example:

```
>>> import turtle
>>> turtle.goto(100, 150)
>>> turtle.pos()
(100.00, 150.00)
>>>
```

You can also use the `turtle.xcor()` command to display the turtle's X coordinate, and the `turtle.ycor()` command to display the turtle's Y coordinate. Here is an example:

```
>>> import turtle
>>> turtle.goto(100, 150)
>>> turtle.xcor()
100
>>> turtle.ycor()
```

```
150
```

```
>>>
```

Controlling the Turtle's Animation Speed

You can use the `turtle.speed(speed)` command to change the speed at which the turtle moves. The `speed` argument is a number in the range of 0 through 10. If you specify 0, then the turtle will make all of its moves instantly (animation is disabled). For example, the following interactive session disables the turtle's animation, then draws a circle. As a result, the circle will be instantly drawn:

```
>>> import turtle
>>> turtle.speed(0)
>>> turtle.circle(100)
>>>
```

If you specify a `speed` value in the range of 1 through 10, then 1 is the slowest speed, and 10 is the fastest speed. The following interactive session sets the animation speed to 1 (the slowest speed) then draws a circle:

```
>>> import turtle
>>> turtle.speed(1)
>>> turtle.circle(100)
>>>
```

You can get the current animation speed with the `turtle.speed()` command (do not specify a `speed` argument). Here is an example:

```
>>> import turtle
>>> turtle.speed()
3
>>>
```

Hiding the Turtle

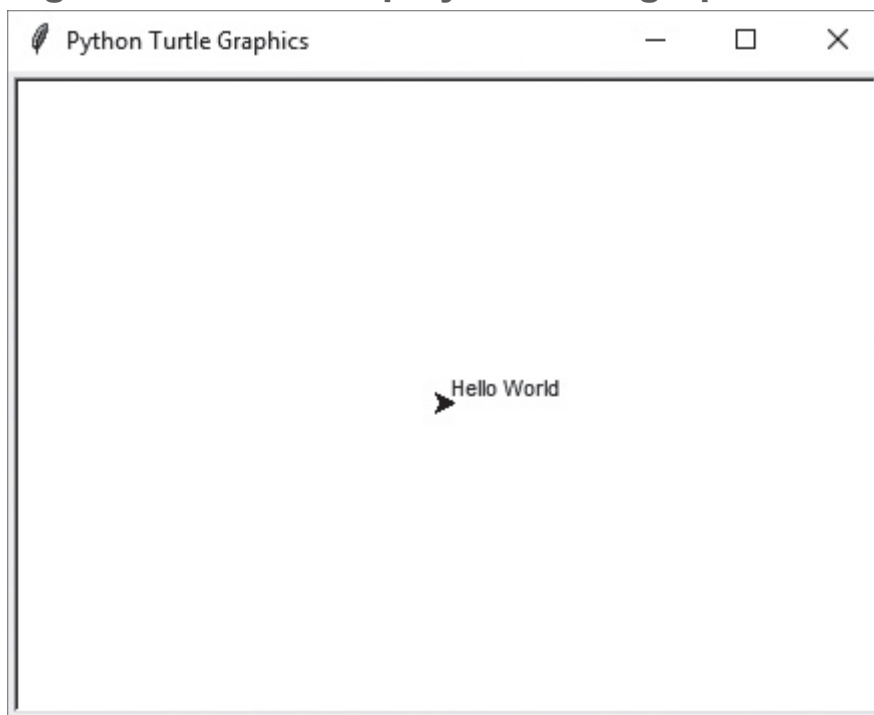
If you don't want the turtle to be displayed, you can use the `turtle.hideturtle()` command to hide it. This command does not change the way graphics are drawn, it simply hides the turtle icon. When you want to display the turtle again, use the `turtle.showturtle()` command.

Displaying Text in the Graphics

Window

You can use the `turtle.write(text)` command to display text in the graphics window. The `text` argument is a string that you want to display. When the string is displayed, the lower-left corner of the first character will be positioned at the turtle's *X* and *Y* coordinates. The following interactive session demonstrates this. The session's output is shown in **Figure 2-22** .

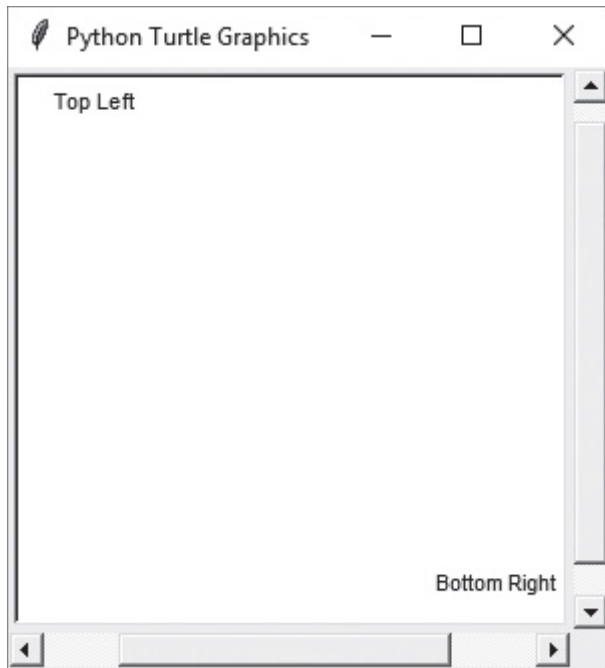
Figure 2-22 Text displayed in the graphics window



```
>>> import turtle
>>> turtle.write('Hello World')
>>>
```

The following interactive session shows another example in which the turtle is moved to specific locations to display text. The session's output is shown in [Figure 2-23](#).

Figure 2-23 Text displayed at specific locations in the graphics window

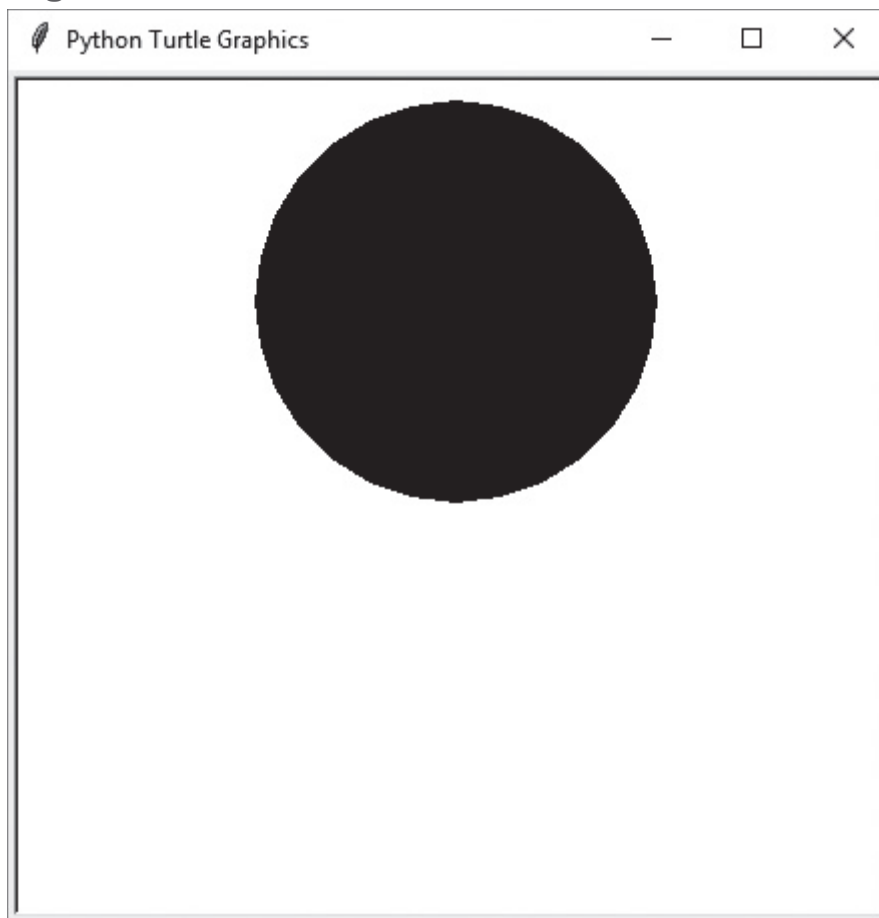


```
>>> import turtle
>>> turtle.setup(300, 300)
>>> turtle.penup()
>>> turtle.hideturtle()
>>> turtle.goto(-120, 120)
>>> turtle.write("Top Left")
>>> turtle.goto(70, -120)
>>> turtle.write("Bottom Right")
>>>
```

Filling Shapes

To fill a shape with a color, you use the `turtle.begin_fill()` command before drawing the shape, then you use the `turtle.end_fill()` command after the shape is drawn. When the `turtle.end_fill()` command executes, the shape will be filled with the current fill color. The following interactive session demonstrates this. The session's output is shown in [Figure 2-24](#) .

Figure 2-24 A filled circle




```
>>> import turtle
>>> turtle.hideturtle()
>>> turtle.begin_fill()
>>> turtle.circle(100)
>>> turtle.end_fill()
>>>
```

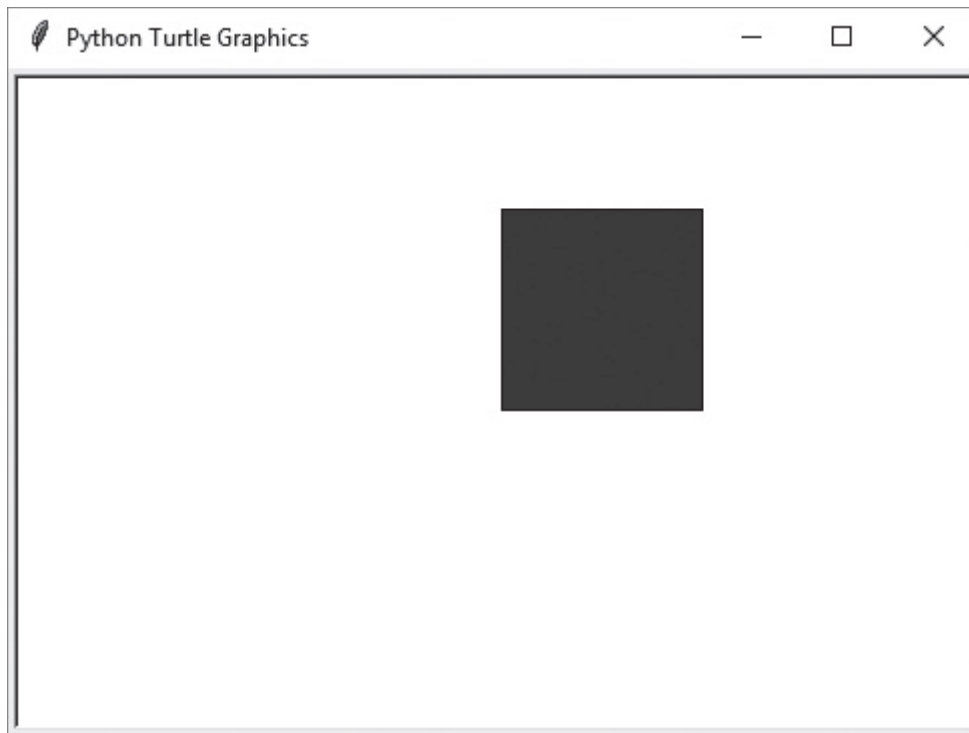
The circle that is drawn in [Figure 2-24](#) is filled with black, which is the default color. You can change the fill color with the `turtle.fillcolor(color)` command. The `color` argument is the name of a color, as a string. For example, the following interactive session changes the drawing color to red, then draws a circle:

```
>>> import turtle
>>> turtle.hideturtle()
>>> turtle.fillcolor('red')
>>> turtle.begin_fill()
>>> turtle.circle(100)
>>> turtle.end_fill()
>>>
```

There are numerous predefined color names that you can use with the `turtle.fillcolor` command, and [Appendix D](#) shows the complete list. Some of the more common colors are `'red'`, `'green'`, `'blue'`, `'yellow'`, and `'cyan'`.

The following interactive session demonstrates how to draw a square that is filled with the color blue. The session's output is shown in **Figure 2-25** .

Figure 2-25 A filled square

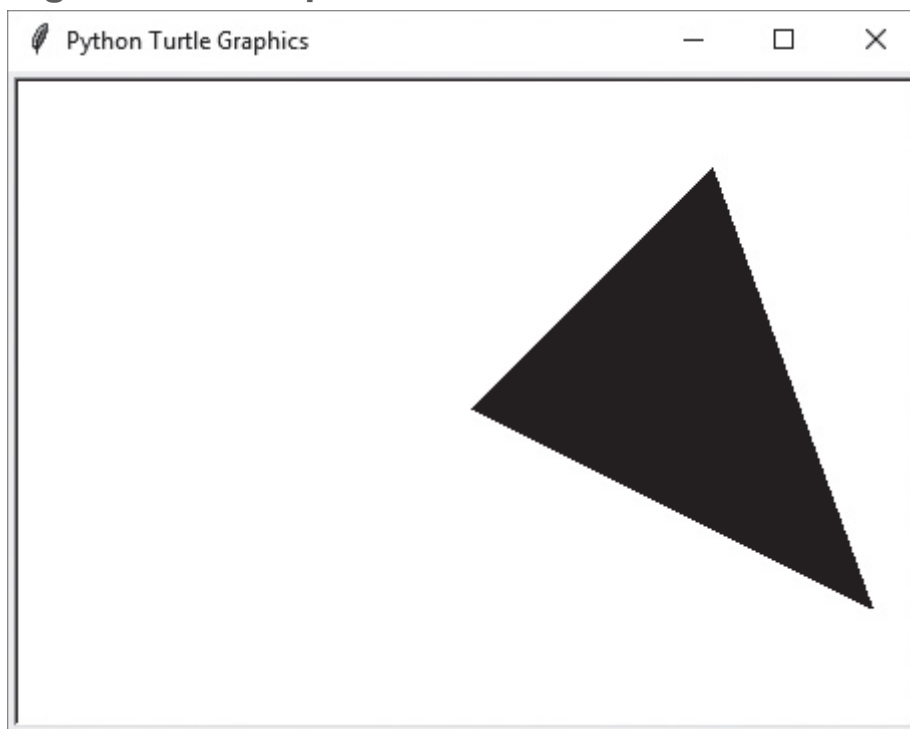


```
>>> import turtle
>>> turtle.hideturtle()
>>> turtle.fillcolor('blue')
>>> turtle.begin_fill()
>>> turtle.forward(100)
>>> turtle.left(90)
>>> turtle.forward(100)
>>> turtle.left(90)
>>> turtle.forward(100)
```

```
>>> turtle.left(90)
>>> turtle.forward(100)
>>> turtle.end_fill()
>>>
```

If you fill a shape that is not enclosed, the shape will be filled as if you had drawn a line connecting the starting point with the ending point. For example, the following interactive session draws two lines. The first is from (0, 0) to (120, 120), and the second is from (120, 120) to (200, -100). When the `turtle.end_fill()` command executes, the shape is filled as if there were a line from (0, 0) to (200, -120). **Figure 2-26**  shows the session's output.

Figure 2-26 Shape filled



```
>>> import turtle
>>> turtle.hideturtle()
>>> turtle.begin_fill()
>>> turtle.goto(120, 120)
>>> turtle.goto(200, -100)
>>> turtle.end_fill()
>>>
```

Using `turtle.done()` to Keep the Graphics Window Open

If you are running a Python turtle graphics program from an environment other than IDLE (for example, at the command line), you may notice the graphics window disappears as soon as your program ends. To prevent the window from closing after the program ends, you will need to add the `turtle.done()` statement to the very end of your turtle graphics programs. This will cause the graphics window to remain open, so you can see its contents after the program finishes executing. To close the window, simply click the window's standard "close" button.

If you are running your programs from IDLE, it is not necessary to have the `turtle.done()` statement in your programs.



In the

Spotlight: The Orion Constellation

Program

Orion is one of the most famous constellations in the night sky. The diagram in [Figure 2-27](#)  shows the approximate positions of several stars in the constellation. The topmost stars are Orion's shoulders, the row of three stars in the middle are Orion's belt, and the bottom two stars are Orion's knees. The diagram in [Figure 2-28](#)  shows the names of each of these stars, and [Figure 2-29](#)  shows the lines that are typically used to connect the stars.

Figure 2-27 Stars in the Orion constellation

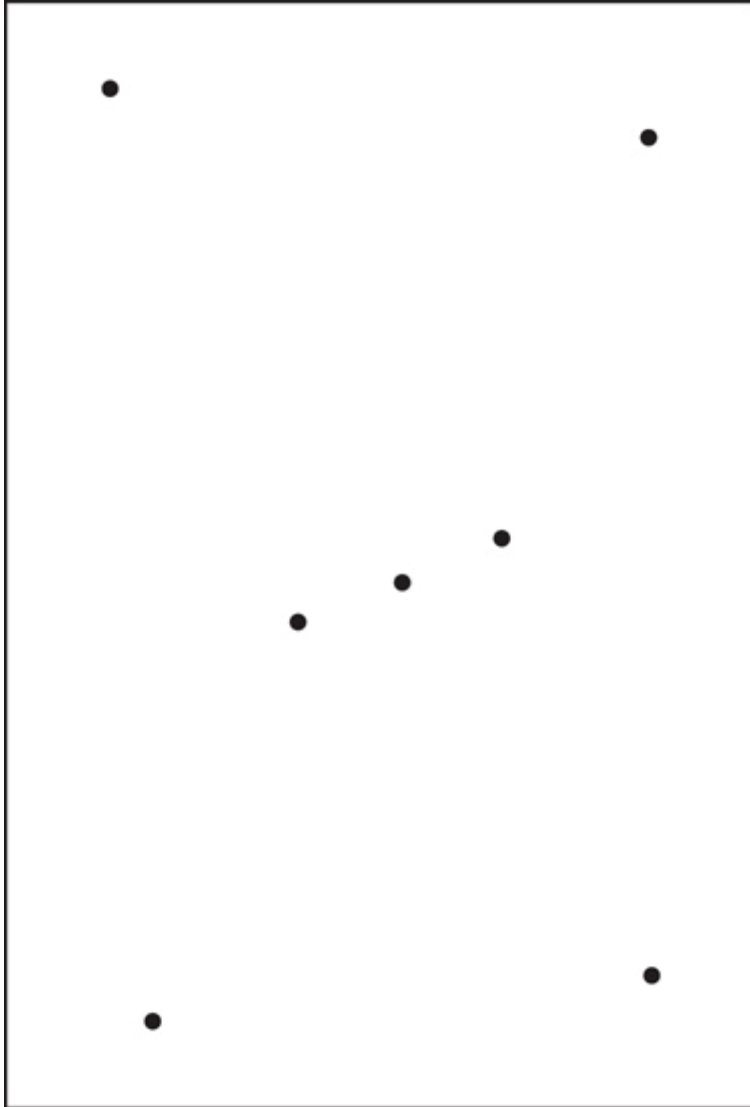


Figure 2-28 Names of the stars

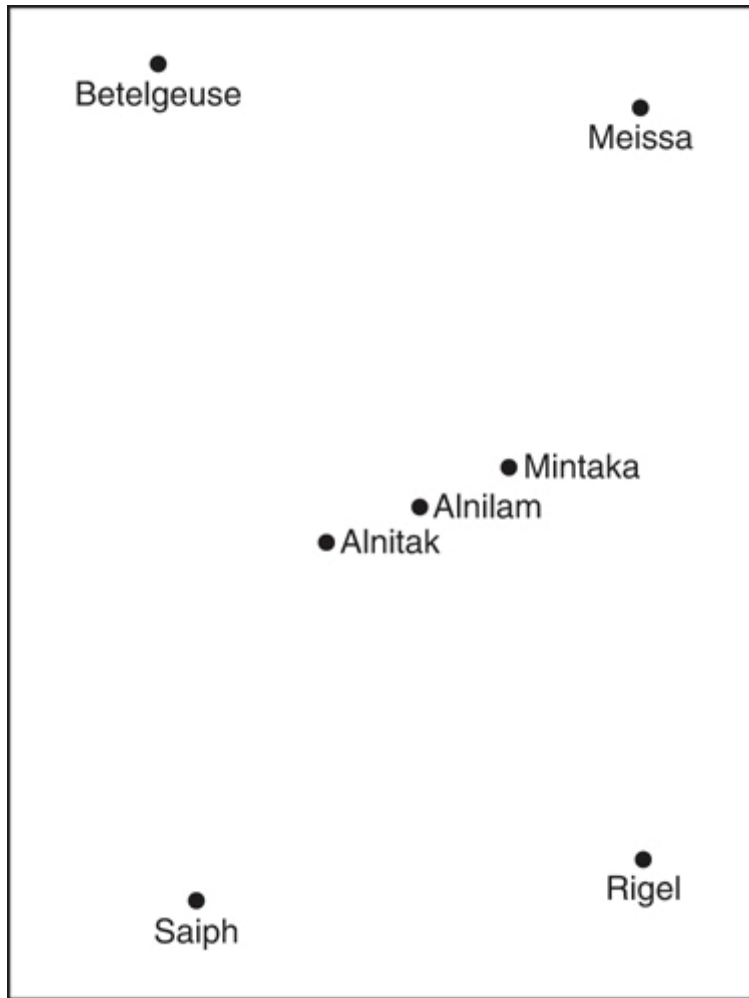
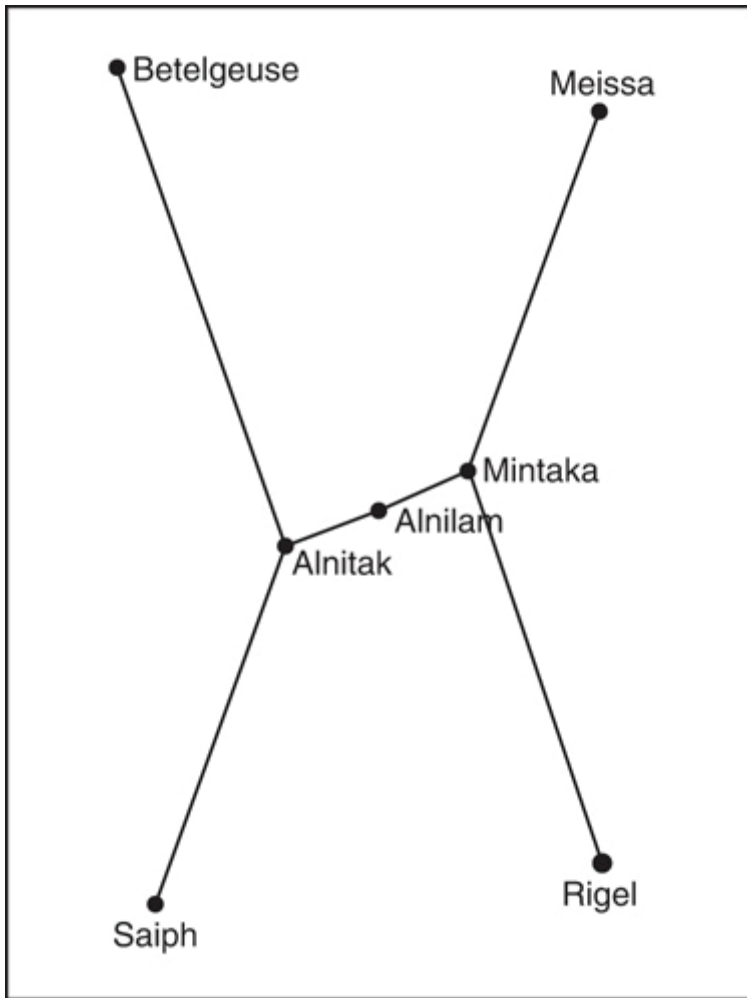
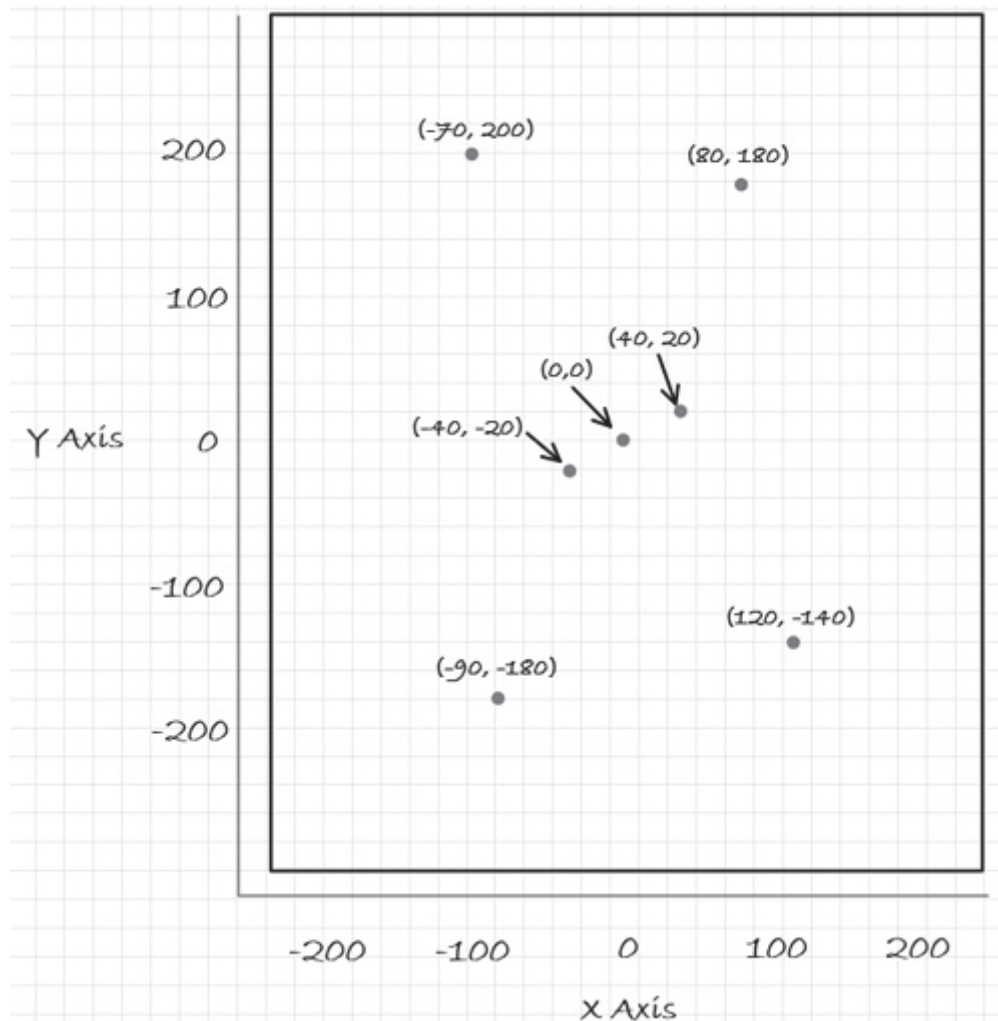


Figure 2-29 Constellation lines



In this section, we will develop a program that displays the stars, the star names, and the constellation lines as they are shown in [Figure 2-29](#) . The program will display the constellation in a graphics window that is 500 pixels wide and 600 pixels high. The program will display dots to represent the stars. We will use a piece of graph paper, as shown in [Figure 2-30](#) , to sketch the positions of the dots and determine their coordinates.

Figure 2-30 Hand sketch of the Orion constellation



We will be using the coordinates that are identified in **Figure 2-30** multiple times in our program. As you can imagine, keeping track of the correct coordinates for each star can be difficult and tedious. To make things more simple in our code, we will create the following named constants to represent each star's coordinates:

```
LEFT_SHOULDER_X = -70
LEFT_SHOULDER_Y = 200
```

```

RIGHT_SHOULDER_X = 80
RIGHT_SHOULDER_Y = 180

LEFT_BELTSTAR_X = -40
LEFT_BELTSTAR_Y = -20

MIDDLE_BELTSTAR_X = 0
MIDDLE_BELTSTAR_Y = 0

RIGHT_BELTSTAR_X = 40
RIGHT_BELTSTAR_Y = 20

LEFT_KNEE_X = -90
LEFT_KNEE_Y = -180

RIGHT_KNEE_X = 120
RIGHT_KNEE_Y = -140

```

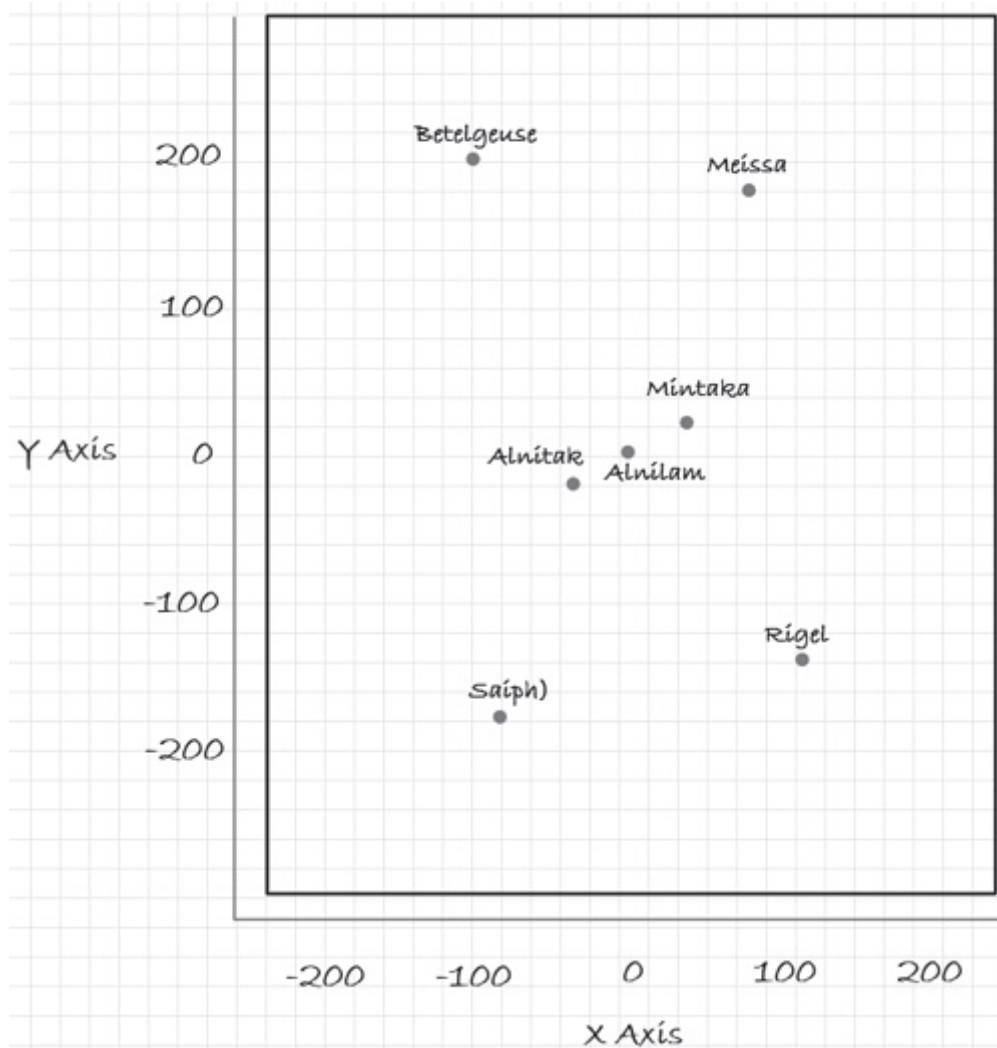
Now that we have identified the coordinates for the stars and created named constants to represent them, we can write pseudocode for the first part of the program, which displays the stars:

<i>Set the graphics window size to 500 pixels wide by 600 pixels high</i>	
<i>Draw a dot at (LEFT_SHOULDER_X, LEFT_SHOULDER_Y)</i>	<i># Left shoulder</i>
<i>Draw a dot at (RIGHT_SHOULDER_X,</i>	<i># Right shoulder</i>

<i>RIGHT_SHOULDER_Y)</i>	
<i>Draw a dot at (LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)</i>	<i># Leftmost star in the belt</i>
<i>Draw a dot at (MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)</i>	<i># Middle star in the belt</i>
<i>Draw a dot at (RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)</i>	<i># Rightmost star in the belt</i>
<i>Draw a dot at (LEFT_KNEE_X, LEFT_KNEE_Y)</i>	<i># Left knee</i>
<i>Draw a dot at (RIGHT_KNEE_X, RIGHT_KNEE_Y)</i>	<i># Right knee</i>

Next, we will display the names of each star, as sketched in **Figure 2-31** . The pseudocode for displaying these names follows.

Figure 2-31 Orion sketch with the names of the stars

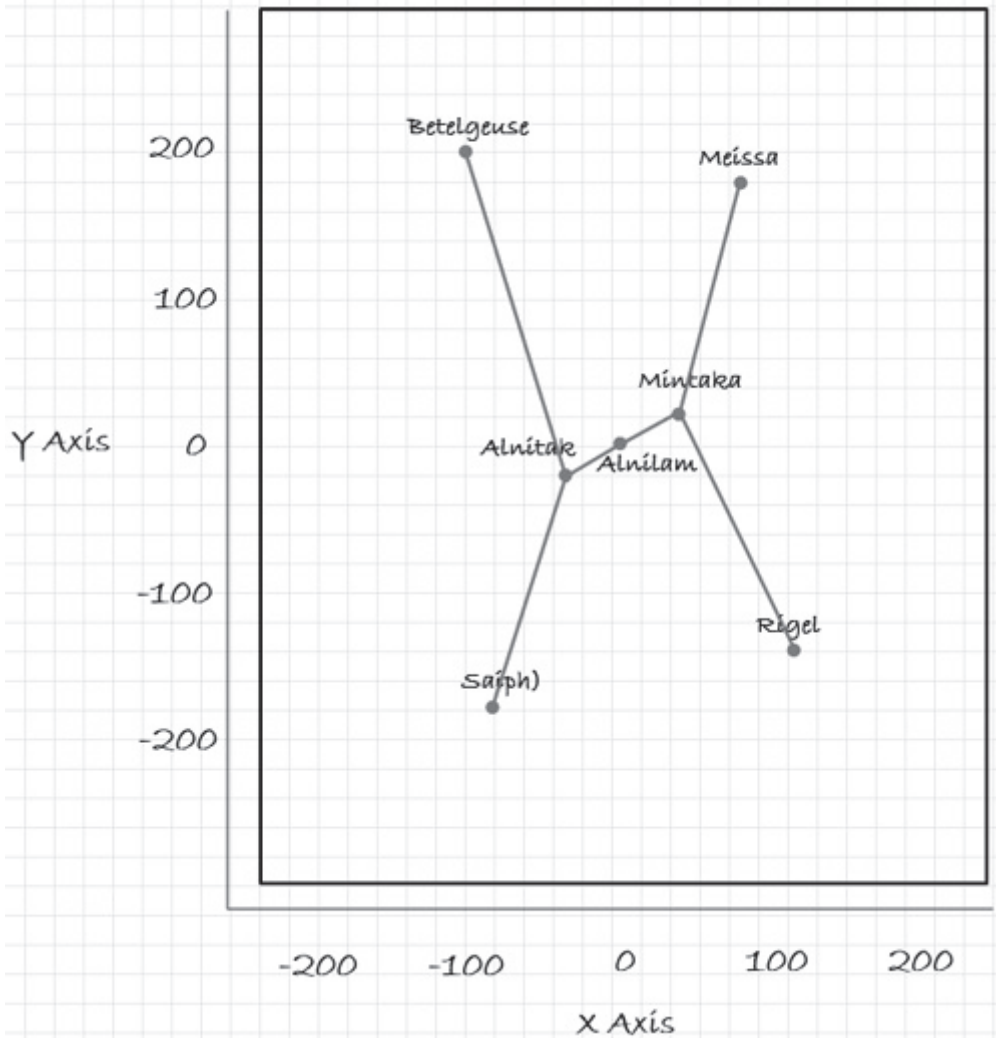


Display the text "Betelgeuse" at (LEFT_SHOULDER_X, LEFT_SHOULDER_Y)	# Left shoulder
Display the text "Meissa" at (RIGHT_SHOULDER_X, RIGHT_SHOULDER_Y)	# Right shoulder
Display the text "Alnitak" at (LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)	# Leftmost star in the belt
Display the text "Alnilam" at (MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)	# Middle star in the belt

<i>Display the text “Mintaka” at (RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)</i>	<i># Rightmost star in the belt</i>
<i>Display the text “Saiph” at (LEFT_KNEE_X, LEFT_KNEE_Y)</i>	<i># Left knee</i>
<i>Display the text “Rigel” at (RIGHT_KNEE_X, RIGHT_KNEE_Y)</i>	<i># Right knee</i>

Next, we will display the lines that connect the stars, as sketched in [Figure 2-32](#) . The pseudocode for displaying these lines follows.

Figure 2-32 Orion sketch with the names of the stars and constellation lines



Left shoulder to left belt star

Draw a line from (LEFT_SHOULDER_X, LEFT_SHOULDER_Y) to
(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)

Right shoulder to right belt star

Draw a line from (RIGHT_SHOULDER_X, RIGHT_SHOULDER_Y) to
(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)

Left belt star to middle belt star

*Draw a line from (LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y) to
(MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)*

Middle belt star to right belt star

*Draw a line from (MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y) to
(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)*

Left belt star to left knee

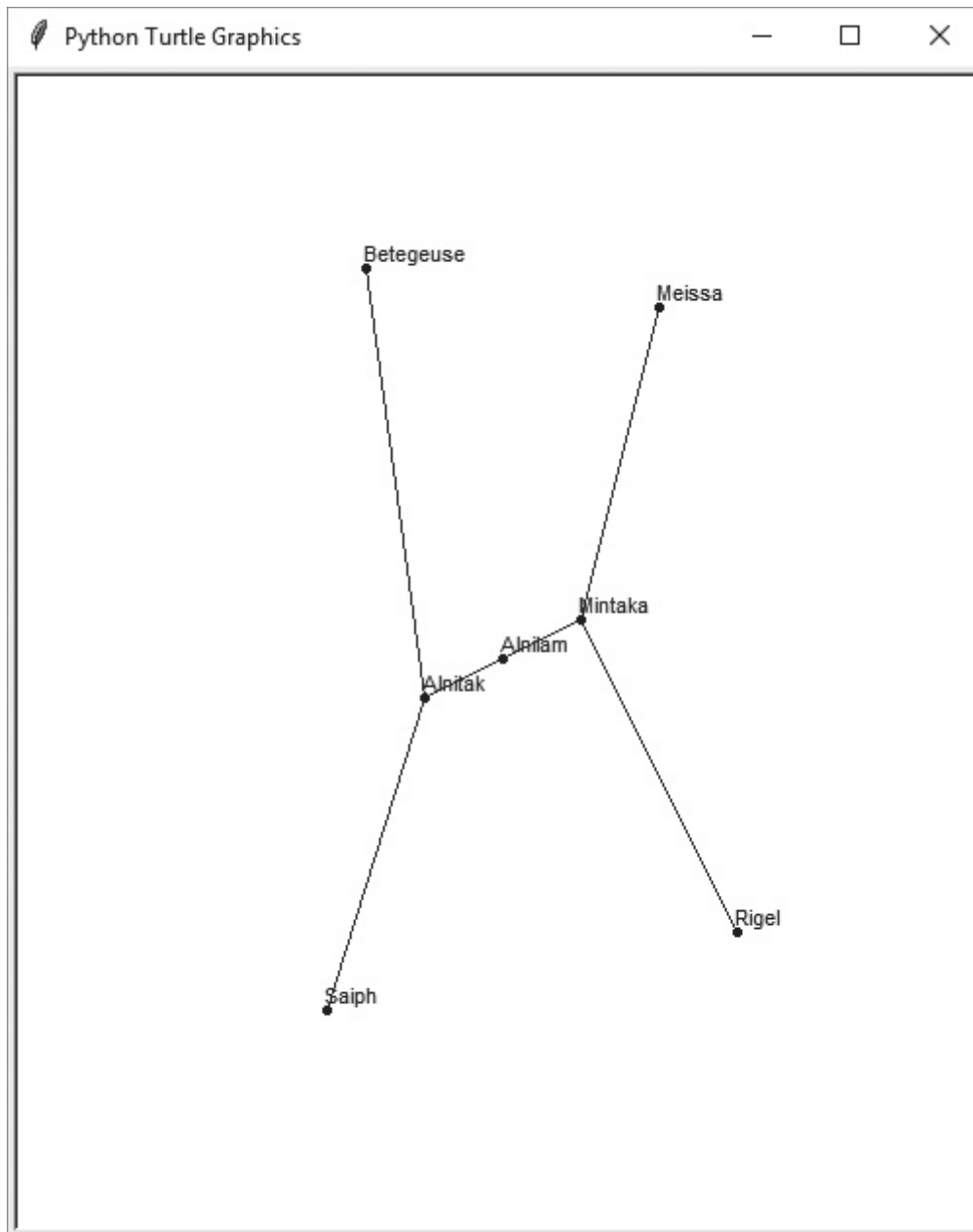
*Draw a line from (LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y) to
(LEFT_KNEE_X, LEFT_KNEE_Y)*

Right belt star to right knee

*Draw a line from (RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y) to
(RIGHT_KNEE_X, RIGHT_KNEE_Y)*

Now that we know the logical steps that the program must perform, we are ready to start writing code. **Program 2-23**  shows the entire program. When the program runs, it first displays the stars, then it displays the names of the stars, and then it displays the constellation lines. **Figure 2-33**  shows the program's output.

Figure 2-33 Output of the orion.py program



Program 2-23 (*orion.py*)

```
1  # This program draws the stars of the Orion  
constellation,  
2  # the names of the stars, and the constellation
```



```
lines.  
3  import turtle  
4  
5  # Set the window size.  
6  turtle.setup(500, 600)  
7  
8  # Setup the turtle.  
9  turtle.penup()  
10 turtle.hideturtle()  
11  
12 # Create named constants for the star coordinates.  
13 LEFT_SHOULDER_X = -70  
14 LEFT_SHOULDER_Y = 200  
15  
16 RIGHT_SHOULDER_X = 80  
17 RIGHT_SHOULDER_Y = 180  
18  
19 LEFT_BELTSTAR_X = -40  
20 LEFT_BELTSTAR_Y = -20  
21  
22 MIDDLE_BELTSTAR_X = 0  
23 MIDDLE_BELTSTAR_Y = 0  
24  
25 RIGHT_BELTSTAR_X = 40  
26 RIGHT_BELTSTAR_Y = 20  
27  
28 LEFT_KNEE_X = -90  
29 LEFT_KNEE_Y = -180
```

```
30
31 RIGHT_KNEE_X = 120
32 RIGHT_KNEE_Y = -140
33
34 # Draw the stars.
35 turtle.goto(LEFT_SHOULDER_X, LEFT_SHOULDER_Y) #
Left shoulder
36 turtle.dot()
37 turtle.goto(RIGHT_SHOULDER_X, RIGHT_SHOULDER_Y)
# Right shoulder
38 turtle.dot()
39 turtle.goto(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y) #
Left belt star
40 turtle.dot()
41 turtle.goto(MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)
# Middle belt star
42 turtle.dot()
43 turtle.goto(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)
# Right belt star
44 turtle.dot()
45 turtle.goto(LEFT_KNEE_X, LEFT_KNEE_Y) #
Left knee
46 turtle.dot()
47 turtle.goto(RIGHT_KNEE_X, RIGHT_KNEE_Y)
# Right knee
48 turtle.dot()
49
50 # Display the star names
```

```
51  turtle.goto(LEFT_SHOULDER_X, LEFT_SHOULDER_Y)  #
Left shoulder
52  turtle.write('Betegeuse')
53  turtle.goto(RIGHT_SHOULDER_X, RIGHT_SHOULDER_Y)
# Right shoulder
54  turtle.write('Meissa')
55  turtle.goto(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)  #
Left belt star
56  turtle.write('Alnitak')
57  turtle.goto(MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)
# Middle belt star
58  turtle.write('Alnilam')
59  turtle.goto(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)
# Right belt star
60  turtle.write('Mintaka')
61  turtle.goto(LEFT_KNEE_X, LEFT_KNEE_Y)  #
Left knee
62  turtle.write('Saiph')
63  turtle.goto(RIGHT_KNEE_X, RIGHT_KNEE_Y)
# Right knee
64  turtle.write('Rigel')
65
66  # Draw a line from the left shoulder to left belt
star
67  turtle.goto(LEFT_SHOULDER_X, LEFT_SHOULDER_Y)
68  turtle.pendown()
69  turtle.goto(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)
70  turtle.penup()
```

```
71
72 # Draw a line from the right shoulder to right
belt star
73 turtle.goto(RIGHT_SHOULDER_X, RIGHT_SHOULDER_Y)
74 turtle.pendown()
75 turtle.goto(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)
76 turtle.penup()
77
78 # Draw a line from the left belt star to middle
belt star
79 turtle.goto(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)
80 turtle.pendown()
81 turtle.goto(MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)
82 turtle.penup()
83
84 # Draw a line from the middle belt star to right
belt star
85 turtle.goto(MIDDLE_BELTSTAR_X, MIDDLE_BELTSTAR_Y)
86 turtle.pendown()
87 turtle.goto(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)
88 turtle.penup()
89
90 # Draw a line from the left belt star to left knee
91 turtle.goto(LEFT_BELTSTAR_X, LEFT_BELTSTAR_Y)
92 turtle.pendown()
93 turtle.goto(LEFT_KNEE_X, LEFT_KNEE_Y)
94 turtle.penup()
95
```

```
96 # Draw a line from the right belt star to right
knee
97 turtle.goto(RIGHT_BELTSTAR_X, RIGHT_BELTSTAR_Y)
98 turtle.pendown()
99 turtle.goto(RIGHT_KNEE_X, RIGHT_KNEE_Y)
100
101 # Keep the window open. (Not necessary with IDLE.)
102 turtle.done()
```



Checkpoint

- 2.30 What is the turtle's default heading when it first appears?
- 2.31 How do you move the turtle forward?
- 2.32 How would you turn the turtle right by 45 degrees?
- 2.33 How would you move the turtle to a new location without drawing a line?
- 2.34 What command would you use to display the turtle's current heading?
- 2.35 What command would you use to draw a circle with a radius of 100 pixels?
- 2.36 What command would you use to change the turtle's pen size to 8 pixels?
- 2.37 What command would you use to change the turtle's drawing color to blue?
- 2.38 What command would you use to change the background color of the turtle's graphics window to black?

2.39 What command would you use to set the size of the turtle's graphics window to 500 pixels wide by 200 pixels high?

2.40 What command would you use to move the turtle to the location (100, 50)?

2.41 What command would you use to display the coordinates of the turtle's current position?

2.42 Which of the following commands will make the animation speed faster? `turtle.speed(1)` or `turtle.speed(10)`

2.43 What command would you use to disable the turtle's animation?

2.44 Describe how to draw a shape that is filled with a color.

2.45 How do you display text in the turtle's graphics window?

Review Questions

Multiple Choice

1. A _____ error does not prevent the program from running, but causes it to produce incorrect results.
 - a. syntax
 - b. hardware
 - c. logic
 - d. fatal

2. A _____ is a single function that the program must perform in order to satisfy the customer.
 - a. task
 - b. software requirement
 - c. prerequisite
 - d. predicate

3. A(n) _____ is a set of well-defined logical steps that must be taken to perform a task.
 - a. logarithm
 - b. plan of action
 - c. logic schedule
 - d. algorithm

4. An informal language that has no syntax rules and is not meant to be compiled or executed is called _____.
- a. faux code
 - b. pseudocode
 - c. Python
 - d. a flowchart
5. A _____ is a diagram that graphically depicts the steps that take place in a program.
- a. flowchart
 - b. step chart
 - c. code graph
 - d. program graph
6. A _____ is a sequence of characters.
- a. char sequence
 - b. character collection
 - c. string
 - d. text block
7. A _____ is a name that references a value in the computer's memory.
- a. variable
 - b. register
 - c. RAM slot
 - d. byte

8. A _____ is any hypothetical person using a program and providing input for it.
- a. designer
 - b. user
 - c. guinea pig
 - d. test subject
9. A string literal in Python must be enclosed in _____.
- a. parentheses.
 - b. single-quotes.
 - c. double-quotes.
 - d. either single-quotes or double-quotes.
10. Short notes placed in different parts of a program explaining how those parts of the program work are called _____.
- a. comments
 - b. reference manuals
 - c. tutorials
 - d. external documentation
11. A(n) _____ makes a variable reference a value in the computer's memory.
- a. variable declaration
 - b. assignment statement
 - c. math expression
 - d. string literal
12. This symbol marks the beginning of a comment in Python.

a. `&`

b. `*`

c. `**`

d. `#`

13. Which of the following statements will cause an error?

a. `x = 17`

b. `17 = x`

c. `x = 99999`

d. `x = '17'`

14. In the expression $12 + 7$, the values on the right and left of the $+$ symbol are called _____.

a. operands

b. operators

c. arguments

d. math expressions

15. This operator performs integer division.

a. `//`

b. `%`

c. `**`

d. `/`

16. This is an operator that raises a number to a power.

a. `%`

b. `*`

c. `**`

d. `/`

17. This operator performs division, but instead of returning the quotient it returns the remainder.

a. `%`

b. `*`

c. `**`

d. `/`

18. Suppose the following statement is in a program: `price = 99.0`. After this statement executes, the price variable will reference a value of which data type?

a. `int`

b. `float`

c. `currency`

d. `str`

19. Which built-in function can be used to read input that has been typed on the keyboard?

a. `input()`

b. `get_input()`

c. `read_input()`

d. `keyboard()`

20. Which built-in function can be used to convert an `int` value to a `float`?

- a. `int_to_float()`
- b. `float()`
- c. `convert()`
- d. `int()`

21. A magic number is _____.

- a. a number that is mathematically undefined
- b. an unexplained value that appears in a program's code
- c. a number that cannot be divided by 1
- d. a number that causes computers to crash

22. A _____ is a name that represents a value that does not change during the program's execution.

- a. named literal
- b. named constant
- c. variable signature
- d. key term

True or False

1. Programmers must be careful not to make syntax errors when writing pseudocode programs.
2. In a math expression, multiplication and division take place before addition and subtraction.

3. Variable names can have spaces in them.
4. In Python, the first character of a variable name cannot be a number.
5. If you print a variable that has not been assigned a value, the number 0 will be displayed.

Short Answer

1. What does a professional programmer usually do first to gain an understanding of a problem?
2. What is pseudocode?
3. Computer programs typically perform what three steps?
4. If a math expression adds a `float` to an `int`, what will the data type of the result be?
5. What is the difference between floating-point division and integer division?
6. What is a magic number? Why are magic numbers problematic?
7. Assume a program uses the named constant `PI` to represent the value 3.14159. The program uses the named constant in several statements. What is the advantage of using the named constant instead of the actual value 3.14159 in each statement?

Algorithm Workbench

1. Write Python code that prompts the user to enter his or her height and assigns the user's input to a variable named `height`.
2. Write Python code that prompts the user to enter his or her favorite color and assigns the user's input to a variable named `color`.
3. Write assignment statements that perform the following operations with the variables `a`, `b`, and `c`:
 - a. Adds 2 to `a` and assigns the result to `b`
 - b. Multiplies `b` times 4 and assigns the result to `a`
 - c. Divides `a` by 3.14 and assigns the result to `b`
 - d. Subtracts 8 from `b` and assigns the result to `a`
4. Assume the variables `result`, `w`, `x`, `y`, and `z` are all integers, and that `w = 5`, `x = 4`, `y = 8`, and `z = 2`. What value will be stored in `result` after each of the following statements execute?
 - a. `result = x + y`
 - b. `result = z * 2`
 - c. `result = y / x`
 - d. `result = y - z`
 - e. `result = w // z`
5. Write a Python statement that assigns the sum of 10 and 14 to the variable `total`.
6. Write a Python statement that subtracts the variable `down_payment` from the variable `total` and assigns the result to

the variable `due`.

7. Write a Python statement that multiplies the variable `subtotal` by 0.15 and assigns the result to the variable `total`.
8. What would the following display?

```
a = 5
b = 2
c = 3
result = a + b * c
print(result)
```

9. What would the following display?

```
num = 99
num = 5
print(num)
```

10. Assume the variable `sales` references a `float` value. Write a statement that displays the value rounded to two decimal points.
11. Assume the following statement has been executed:

```
number = 1234567.456
```

Write a Python statement that displays the value referenced by the `number` variable formatted as

```
1,234,567.5
```

12. What will the following statement display?

```
print('George', 'John', 'Paul', 'Ringo', sep='@')
```

13. Write a turtle graphics statement that draws a circle with a radius of 75 pixels.
14. Write the turtle graphics statements to draw a square that is 100 pixels wide on each side and filled with the color blue.
15. Write the turtle graphics statements to draw a square that is 100 pixels wide on each side and a circle that is centered inside the square. The circle's radius should be 80 pixels. The circle should be filled with the color red. (The square should not be filled with a color.)

Programming Exercises

1. Personal Information

Write a program that displays the following information:

- Your name
- Your address, with city, state, and ZIP
- Your telephone number
- Your college major

2. Sales Prediction

A company has determined that its annual profit is typically 23 percent of total sales. Write a program that asks the user to enter the projected amount of total sales, then displays the profit that will be made from that amount.



The Sales Prediction Problem

Hint: Use the value 0.23 to represent 23 percent.

3. Land Calculation

One acre of land is equivalent to 43,560 square feet. Write a program that asks the user to enter the total square feet in a tract of land and calculates the number of acres in the tract. Hint: Divide the amount entered by 43,560 to get the number of acres.

4. **Total Purchase**

A customer in a store is purchasing five items. Write a program that asks for the price of each item, then displays the subtotal of the sale, the amount of sales tax, and the total. Assume the sales tax is 7 percent.

5. **Distance Traveled**

Assuming there are no accidents or delays, the distance that a car travels down the interstate can be calculated with the following formula:

$$\text{Distance} = \text{Speed} \times \text{Time}$$

A car is traveling at 70 miles per hour. Write a program that displays the following:

- The distance the car will travel in 6 hours
- The distance the car will travel in 10 hours
- The distance the car will travel in 15 hours

6. **Sales Tax**

Write a program that will ask the user to enter the amount of a purchase. The program should then compute the state and county sales tax. Assume the state sales tax is 5 percent and the county sales tax is 2.5 percent. The program should display the amount of the purchase, the state sales tax, the county

sales tax, the total sales tax, and the total of the sale (which is the sum of the amount of purchase plus the total sales tax).

Hint: Use the value 0.025 to represent 2.5 percent, and 0.05 to represent 5 percent.

7. **Miles-per-Gallon**

A car's miles-per-gallon (MPG) can be calculated with the following formula:

$$\text{MPG} = \text{Miles driven} \div \text{Gallons of gas used}$$

Write a program that asks the user for the number of miles driven and the gallons of gas used. It should calculate the car's MPG and display the result.

8. **Tip, Tax, and Total**

Write a program that calculates the total amount of a meal purchased at a restaurant. The program should ask the user to enter the charge for the food, then calculate the amounts of a 18 percent tip and 7 percent sales tax. Display each of these amounts and the total.

9. **Celsius to Fahrenheit Temperature Converter**

Write a program that converts Celsius temperatures to Fahrenheit temperatures. The formula is as follows:

$$F = 95C + 32$$

The program should ask the user to enter a temperature in Celsius, then display the temperature converted to Fahrenheit.

10. **Ingredient Adjuster**

A cookie recipe calls for the following ingredients:

- 1.5 cups of sugar

- 1 cup of butter
- 2.75 cups of flour

The recipe produces 48 cookies with this amount of the ingredients. Write a program that asks the user how many cookies he or she wants to make, then displays the number of cups of each ingredient needed for the specified number of cookies.

11. **Male and Female Percentages**

Write a program that asks the user for the number of males and the number of females registered in a class. The program should display the percentage of males and females in the class.

Hint: Suppose there are 8 males and 12 females in a class. There are 20 students in the class. The percentage of males can be calculated as $8 \div 20 = 0.4$, or 40%. The percentage of females can be calculated as $12 \div 20 = 0.6$, or 60%.

12. **Stock Transaction Program**

Last month, Joe purchased some stock in Acme Software, Inc. Here are the details of the purchase:

- The number of shares that Joe purchased was 2,000.
- When Joe purchased the stock, he paid \$40.00 per share.
- Joe paid his stockbroker a commission that amounted to 3 percent of the amount he paid for the stock.

Two weeks later, Joe sold the stock. Here are the details of the sale:

- The number of shares that Joe sold was 2,000.

- He sold the stock for \$42.75 per share.
- He paid his stockbroker another commission that amounted to 3 percent of the amount he received for the stock.

Write a program that displays the following information:

- The amount of money Joe paid for the stock.
- The amount of commission Joe paid his broker when he bought the stock.
- The amount for which Joe sold the stock.
- The amount of commission Joe paid his broker when he sold the stock.
- Display the amount of money that Joe had left when he sold the stock and paid his broker (both times). If this amount is positive, then Joe made a profit. If the amount is negative, then Joe lost money.

13. **Planting Grapevines**

A vineyard owner is planting several new rows of grapevines, and needs to know how many grapevines to plant in each row. She has determined that after measuring the length of a future row, she can use the following formula to calculate the number of vines that will fit in the row, along with the trellis end-post assemblies that will need to be constructed at each end of the row:

$$V=R-2ES$$

The terms in the formula are:

V is the number of grapevines that will fit in the row.

R is the length of the row, in feet.

E is the amount of space, in feet, used by an end-post assembly.

S is the space between vines, in feet.

Write a program that makes the calculation for the vineyard owner. The program should ask the user to input the following:

- The length of the row, in feet
- The amount of space used by an end-post assembly, in feet
- The amount of space between the vines, in feet

Once the input data has been entered, the program should calculate and display the number of grapevines that will fit in the row.

14. **Compound Interest**

When a bank account pays compound interest, it pays interest not only on the principal amount that was deposited into the account, but also on the interest that has accumulated over time. Suppose you want to deposit some money into a savings account, and let the account earn compound interest for a certain number of years. The formula for calculating the balance of the account after a specified number of years is:

$$A = P(1 + rn)^{nt}$$

The terms in the formula are:

A is the amount of money in the account after the specified number of years.

P is the principal amount that was originally deposited into the account.

r is the annual interest rate.

n is the number of times per year that the interest is compounded.

t is the specified number of years.

Write a program that makes the calculation for you. The program should ask the user to input the following:

- The amount of principal originally deposited into the account
- The annual interest rate paid by the account
- The number of times per year that the interest is compounded (For example, if interest is compounded monthly, enter 12. If interest is compounded quarterly, enter 4.)
- The number of years the account will be left to earn interest

Once the input data has been entered, the program should calculate and display the amount of money that will be in the account after the specified number of years.



Note:

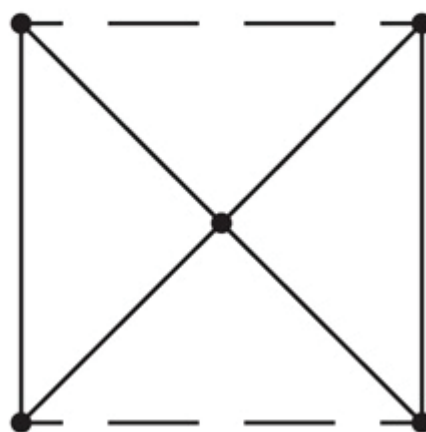
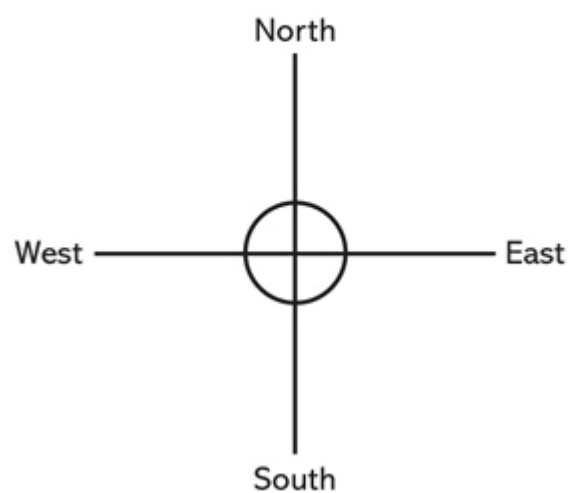
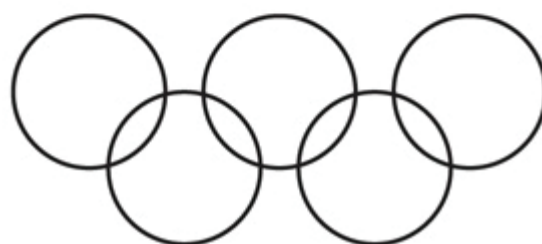
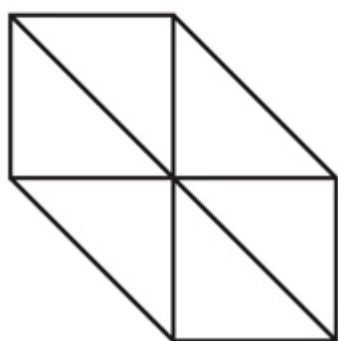
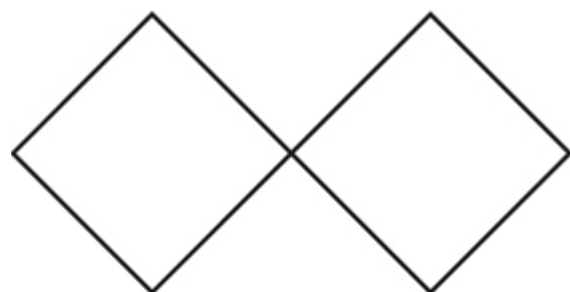
The user should enter the interest rate as a percentage. For example, 2 percent would be entered as 2, not as

.02. The program will then have to divide the input by 100 to move the decimal point to the correct position.

15. **Turtle Graphics Drawings**

Use the turtle graphics library to write programs that reproduce each of the designs shown in [Figure 2-34](#) .

Figure 2-34 Designs



Chapter 3 Decision Structures and Boolean Logic

Topics

3.1 The `if` Statement 

3.2 The `if-else` Statement 

3.3 Comparing Strings 

3.4 Nested Decision Structures and the `if-elif-else` Statement 

3.5 Logical Operators 

3.6 Boolean Variables 

3.7 Turtle Graphics: Determining the State of the Turtle 